

core java, advanced java, ani java full stack yamde different kay ahe

हो, तिन्ही Java शी related आहेत, पण **scope वेगळा** आहे. सोप्या भाषेत:

Course	काय शिकतात?	मुख्य उपयोग
Core Java	Java ची basics + OOP + Collections + Exception Handling + Multithreading	Java programming ची strong foundation
Advanced Java	Java वापरून web/database/server-side development	Backend/Web applications
Java Full Stack	Frontend + Backend + Database + Tools	Complete web application बनवणे

1. Core Java 🚫

हा **starting point** आहे.

यामध्ये साधारण:

- Variables, Data Types
- Loops, Arrays
- **OOP** – Class, Object, Inheritance, Polymorphism
- String
- Exception Handling
- Collections
- Multithreading
- File Handling
- Java 8 features

👉 जर Java नवीन असेल तर **Core Java पासून सुरुवात** कर.

2. Advanced Java 🚫

Core Java नंतर web/backend side शिकवतात.

यामध्ये:

- JDBC – Database connectivity
- Servlets

- JSP
- HTTP/Web concepts
- Session & Cookies
- काही courses मध्ये **Spring / Spring Boot** सुद्धा असते

👉 याचा focus मुख्यतः **backend/web development** वर असतो.

3. Java Full Stack 🌀

हा सर्वात **मोठा scope** असलेला track आहे.

साधारणपणे:

Frontend

→ HTML, CSS, JavaScript, React/Angular

Backend

→ Java, Spring, Spring Boot, REST API

Database

→ MySQL / PostgreSQL

Tools

→ Git, GitHub, Maven, Postman इ.

उदा. तू **Shopping Website** बनवतोस:

- Website दिसण्यासाठी → HTML/CSS/React
- Login, orders, payment logic → Java + Spring Boot
- User/order data → MySQL

हे सगळं मिळून **Java Full Stack**.

🎯 Job साठी कोणता?

जर तुझा goal **Java Developer / Full Stack Developer** बनण्याचा असेल तर:

Core Java → Advanced Java/Backend → Spring Boot → Frontend → Database → Projects

म्हणजे **Core Java आणि Full Stack हे alternatives नाहीत** — Core Java हा Full Stack शिकण्यासाठीचा foundation आहे.

हो 👍 जर तुला **Core Java + Advanced Java + Java Full Stack** या सगळ्यावर projects बनवायचे असतील, तर आपण projects **step-by-step difficulty वाढवत** बनवू शकतो.

मी तुला हा roadmap suggest करेन:

● Level 1 — Core Java Projects

1. Student Management System

- Add / Update / Delete / Search Student
- OOP, Collections, Exception Handling

2. Banking System

- Account creation
- Deposit / Withdraw
- Balance checking

3. Employee Management System

- Employee CRUD
- Salary calculation

● Level 2 — Advanced Java Projects

4. Online Book Store

- Java + JDBC + MySQL
- Login/Register
- Books CRUD
- Cart

5. Employee Management Web App

- Servlet + JSP + JDBC + MySQL

6. Online Quiz System

- User login
- Questions
- Score calculation
- Database

● Level 3 — Java Full Stack Projects

7. E-Commerce Website ★

- Frontend: HTML/CSS/JS किंवा React
- Backend: Java + Spring Boot
- Database: MySQL
- REST APIs

- Login/Register
- Products
- Cart
- Orders

8. Job Portal

- Student/Job seeker login
- Company login
- Job posting
- Apply for job
- Admin panel

9. Hospital Management System

- Patient
- Doctor
- Appointment
- Billing
- Admin

💡 माझा सल्ला

सगळे projects एकदम सुरु करू नकोस. एक project पूर्णपणे build करून त्यातून पुढचा project कर.

आपण अगदी practical पद्धतीने करू शकतो:

Project → Requirements → Database → Java code → Backend → Frontend → Testing → GitHub → Resume मध्ये कसा लिहायचा

जर तू beginner असशील तर आपण Student Management System पासून सुरुवात करून पूर्ण project step-by-step बनवूया.

हो, अगदी बरोबर approach आहे 👍

म्हणजे तुझ्याकडे शेवटी 4 projects असतील:

📌 Project Plan

1 Core Java — 1 Project

उदा. **Banking Management System**

यात Core Java concepts वापरू:

- OOP
- Collections
- Exception Handling
- File Handling
- Multithreading इ.

2 Advanced Java — 1 Project

उदा. **Online Book Store**

यात:

- JDBC
- MySQL
- Servlet
- JSP
- Sessions/Cookies
- CRUD operations

3 Java Full Stack — 1 Project

उदा. **Job Portal**

यात:

- Frontend
- Java + Spring Boot
- REST API
- MySQL
- Authentication
- Git/GitHub

4 Final Integrated Project — Core + Advanced + Full Stack ★

हा तुझा **major/project showcase** असेल.

उदा. **Hospital Management System / E-Commerce / Job Portal 2.0**

यामध्ये आपण आधी शिकलेल्या सगळ्या गोष्टी combine करू:

Frontend



HTML + CSS + JavaScript / React



Spring Boot / Java Backend



REST APIs



Business Logic (Core Java)



MySQL Database



JDBC / JPA-Hibernate

 शेवटी तुझ्या Resume मध्ये

Project	Level
Banking Management System	Core Java
Online Book Store	Advanced Java
Job Portal	Java Full Stack

Smart Hospital Management System Major Integrated Project ★

आणि चौथा project फक्त पहिल्या 3 projects ला जोडून बनवायचा नाही. तो एक नवीन, proper real-world application असेल ज्यात Core Java concepts, Advanced Java/backend concepts आणि Full Stack technologies एकत्र वापरू.

तुला हवं असेल तर आपण आत्तापासून **Project 1 (Core Java) → Project 2 → Project 3 → Final Project** असा complete roadmap बनवून प्रत्येक project प्रत्यक्ष code करून तयार करू शकतो.

Perfect 👍 मग आपण **4-project roadmap** follow करूया.

क्रम:

1.  Core Java → Banking Management System
2.  Advanced Java → Online Book Store

3. 🟠 Java Full Stack → Job Portal
4. ⭐ Final Integrated → Real-world project using all three

प्रत्येक project मध्ये आपण **concept → requirements → database → coding → testing → GitHub → resume description** असा complete flow ठेवू.

पहिल्यापासून सुरु करूया: Core Java – Banking Management System.

चला मग सुरु करूया 🚀

🟢 Project 1: Banking Management System — Core Java

हा project **फक्त Core Java** वर ठेवू म्हणजे तुझी Java fundamentals मजबूत होतील.

आपण यात हे features ठेवू:

- 👤 Create Bank Account
- 🏠 Account Number & PIN
- 💰 Deposit Money
- 💳 Withdraw Money
- 💳 Check Balance
- 📄 Account Details
- 🔍 Search Account
- ✎ Update Account
- ✖ Delete Account
- 📋 Transaction History
- 🚪 Exit

Core Java concepts वापरणार:

- Classes & Objects
- Encapsulation
- Inheritance
- Polymorphism
- Abstraction
- Interfaces

- Collections (ArrayList, HashMap)
- Exception Handling
- File Handling
- Methods & Constructors
- Loops & Conditions

Project Structure

BankingManagementSystem/

```

|
|— model/
|   |— Account.java
|   |— Transaction.java
|
|— service/
|   |— BankService.java
|
|— exception/
|   |— AccountNotFoundException.java
|
|— util/
|   |— FileUtil.java
|
|— Main.java

```

आपण copy-paste करून पुढे जाण्यापेक्षा प्रत्येक class का बनवतो, कोणता concept कुठे वापरतो आणि code कसा काम करतो हे समजून project बनवू.

Next step: आधी आपण **requirements + class design** finalize करू, त्यानंतर Account.java पासून coding सुरू करू.

छान 👍 मग **Banking Management System** चा development सुरू करूया.

Step 1 — Basic Class Design

आपल्याला सुरुवातीला 3 मुख्य classes लागतील:

BankingManagementSystem

```
|
|  └─ Account
|    └─ accountNumber
|    └─ accountHolderName
|    └─ phoneNumber
|    └─ pin
|    └─ balance
|
|  └─ Transaction
|    └─ transactionId
|    └─ type
|    └─ amount
|    └─ date
|
|  └─ BankService
|    └─ createAccount()
|    └─ deposit()
|    └─ withdraw()
|    └─ checkBalance()
|    └─ searchAccount()
|    └─ updateAccount()
|    └─ deleteAccount()
```

Step 2 — आपण कोणते Core Java concepts demonstrate करणार?

Concept	Project मध्ये उपयोग
Class & Object	Account, Transaction
Encapsulation	private variables + getters/setters

Concept	Project मध्ये उपयोग
Constructor	Account तयार करताना
Inheritance	पुढच्या stage मध्ये
Polymorphism	Transaction/Account operations मध्ये
Interface	Service design मध्ये
Collections	अनेक accounts store करण्यासाठी
Exception Handling	Invalid withdrawal/account errors
File Handling	Data save करण्यासाठी
Multithreading	पुढच्या stage मध्ये transaction simulation

Development order

Account → Transaction → BankService → Exceptions → File Handling → Main Menu → Testing → Final Project

यामुळे project फक्त चालणार नाही, तर interview मध्ये प्रत्येक Core Java concept explain करता येईल.

छान 👍 मग **Account.java** पासून सुरुवात करूया.

हा class आपल्या project चा base असेल. यात आपण **Encapsulation + Constructor + Getter/Setter** वापरू.

पुढच्या step मध्ये आपण **Account.java** लिहून प्रत्येक line काय करते ते समजून घेऊ, मग **Transaction.java** कडे जाऊ.

चला 👍 **Account.java** तयार करूया.

Account.java

package model;

```
public class Account {

    private long accountNumber;
    private String accountHolderName;
    private String phoneNumber;
    private int pin;
    private double balance;

    // Constructor
    public Account(long accountNumber, String accountHolderName,
        String phoneNumber, int pin, double balance) {

        this.accountNumber = accountNumber;
        this.accountHolderName = accountHolderName;
        this.phoneNumber = phoneNumber;
        this.pin = pin;
        this.balance = balance;
    }

    // Getters
    public long getAccountNumber() {
        return accountNumber;
    }

    public String getAccountHolderName() {
        return accountHolderName;
    }

    public String getPhoneNumber() {
        return phoneNumber;
    }
}
```

```
public int getPin() {  
    return pin;  
}
```

```
public double getBalance() {  
    return balance;  
}
```

// Setters

```
public void setAccountHolderName(String accountHolderName) {  
    this.accountHolderName = accountHolderName;  
}
```

```
public void setPhoneNumber(String phoneNumber) {  
    this.phoneNumber = phoneNumber;  
}
```

```
public void setPin(int pin) {  
    this.pin = pin;  
}
```

```
public void setBalance(double balance) {  
    this.balance = balance;  
}  
}
```

यात काय शिकलो?

1. private variables → Encapsulation

```
private double balance;
```

बाहेरून कोणीही direct balance बदलू शकत नाही.

2. Constructor

```
public Account(...) {  
    ...  
}
```

नवीन Account object तयार करताना values initialize करण्यासाठी.

3. Getter

```
public double getBalance() {  
    return balance;  
}
```

Balance read करण्यासाठी.

4. Setter

```
public void setBalance(double balance) {  
    this.balance = balance;  
}
```

Balance update करण्यासाठी.

⚠ Real banking application मध्ये PIN plain int म्हणून store करणे योग्य नाही. पण हा **Core Java learning project** असल्यामुळे आपण आता concepts समजण्यासाठी simple ठेवत आहोत.

पुढचा भाग

आता आपण **Transaction.java** बनवू. त्यात **Transaction ID, transaction type, amount आणि date/time** ठेवू आणि त्यातून **LocalDateTime** हा Java concept शिकू.

हो 👍 आधी हे clear करणं महत्वाचं आहे. **Java code कुठे लिहायचा आणि कुठे run करायचा** हे setup करूया.

 **माझा recommendation: IntelliJ IDEA**

आपल्या project साठी **IntelliJ IDEA** वापरूया. Java projects साठी beginner-friendly आणि industry मध्येही widely used आहे.

तुला 3 गोष्टी लागतील:

1. **JDK (Java Development Kit)** — Java code compile/run करण्यासाठी
2. **IntelliJ IDEA** — code लिहिण्यासाठी

3. Terminal / IntelliJ Run button — output पाहण्यासाठी

आपण project असा तयार करू

BankingManagementSystem

|

└─ src

└─ model

└─ Account.java

म्हणजे आपण आता दिलेला code:

```
package model;
```

```
public class Account {
```

```
...
```

```
}
```

हा **Account.java** file मध्ये लिहायचा.

पण **Account.java** एकट्याने run केल्यावर output येणार नाही, कारण त्यात main() method नाही.

Output पाहण्यासाठी नंतर आपण:

Main.java

बनवू आणि त्यात:

```
public static void main(String[] args)
```

असेल.

मग IntelliJ मधील ► **Run** button दाबल्यावर खाली Console मध्ये output दिसेल.

● आपला actual flow

Install JDK

↓

Install IntelliJ IDEA

↓

Create Java Project

↓

Create packages/classes



Write Account.java



Write Main.java



► Run



Console मध्ये Output

जर तुझ्या laptop मध्ये Java/IntelliJ अजून install नसेल, तर आपण coding सुरू करण्याआधी JDK + IntelliJ IDEA installation step-by-step करूया.

Perfect 👍 मग आधी **setup** करूया. घाईने project code लिहायला सुरुवात करू नको; environment ready असणं महत्वाचं आहे.

तुझ्या laptop मध्ये खालीलपैकी काय आहे ते सांग:

- A) JDK installed आहे + IntelliJ IDEA installed आहे
- B) JDK installed आहे पण IntelliJ IDEA नाही
- C) दोन्ही installed नाहीत
- D) माहित नाही

फक्त A/B/C/D पैकी एक पाठव. त्यानुसार पुढची exact step देतो.

काही problem नाही 👍 **D = माहित नाही**, तर आधी आपल्या laptop मध्ये Java आहे की नाही ते check करूया.

Step 1: JDK आहे का ते check कर

Windows laptop असेल तर:

1. Keyboard वर **Windows key** दाब.
2. Search मध्ये **Command Prompt** किंवा **cmd** लिही.
3. Command Prompt उघड.
4. त्यात हे type कर:

java -version

Enter दाब.

नंतर हेही type कर:

```
javac -version
```

Enter दाब.

मला काय पाठवायचं?

दोन्ही commands चा जो output येईल त्याचा screenshot इथे upload कर 📷

मग मी तुला सांगेन:

- JDK installed आहे की नाही
- कोणती version आहे
- IntelliJ install करायची गरज आहे का
- आणि पुढे **Banking Management System** कुठे create करायचा

आता काहीही install करू नकोस. आधी screenshot पाठव.

Perfect! 🎉 तुझ्या laptop मध्ये **JDK 17 already properly installed आहे.**

तुझ्या output वरून:

- ✅ Java: **17 LTS**
- ✅ javac: **17**
- ✅ 64-bit Java
- ✅ Java code compile आणि run करण्यासाठी setup ready आहे

आता फक्त IDE पाहिजे, म्हणजे code व्यवस्थित लिहिण्यासाठी. आपण **IntelliJ IDEA** वापरू शकतो.

पुढचा Step — IntelliJ IDEA

आता मला सांग, **IntelliJ IDEA** आधीपासून laptop मध्ये आहे का हे माहित आहे का?

जर माहित नसेल तर Windows Search मध्ये **IntelliJ IDEA** लिहून बघ.

- दिसलं → मला सांग “आहे”
- दिसलं नाही → “नाही”

मग त्यापुढची step करू.

Perfect! 🔥 मग आपलं Java setup पूर्णपणे ready आहे.

आता IntelliJ IDEA मध्ये आपला पहिला Core Java project तयार करूया.

Step 1 — New Project

IntelliJ IDEA open कर.

1. **New Project** वर click कर.
2. **Java** select कर.
3. Project Name:

BankingManagementSystem

4. **JDK → 17** select कर.
 - जर JDK 17 दिसत असेल तर तेच select कर.
5. Build system सध्या **IntelliJ** ठेव.
6. **Create** वर click कर.

Step 2 — Project तयार झाल्यावर

डाव्या बाजूला साधारण असं दिसेल:

BankingManagementSystem

└─ src

src वर **Right Click → New → Package**

Package name:

model

नंतर:

model वर **Right Click → New → Java Class**

Class name:

Account

आता Account.java उघडेल.

त्यामध्ये मी आधी दिलेला Account.java code paste करू.

⚠ **पण अजून Run करू नकोस.** Account.java मध्ये main() method नाही, त्यामुळे output दिसणार नाही.

आधी **Account class तयार कर**. झाल्यावर मला फक्त **"Account.java तयार झाला"** असं सांग. मग आपण Main.java बनवून पहिल्यांदा output पाहू. 🚀

छान! 🎉 आता आपण पहिल्यांदा **code run करून output पाहूया**.

Step 2 — Main.java तयार कर

src वर **Right Click → New → Java Class** कर.

Class name:

Main

आता Main.java मध्ये हा code टाक:

```
import model.Account;
```

```
public class Main {
```

```
    public static void main(String[] args) {
```

```
        Account account = new Account(
```

```
            1001,
```

```
            "Reshma",
```

```
            "9876543210",
```

```
            1234,
```

```
            5000
```

```
        );
```

```
        System.out.println("==== Account Details =====");
```

```
        System.out.println("Account Number : " + account.getAccountNumber());
```

```
        System.out.println("Account Holder : " + account.getAccountHolderName());
```

```
        System.out.println("Phone Number : " + account.getPhoneNumber());
```

```
        System.out.println("Balance : ₹" + account.getBalance());
```

```
    }
```

```
}
```

▶ आता Run कर

Main.java open ठेवून:

Right Click → Run 'Main.main()'

किंवा वरती दिसणाऱ्या  **Run** button वर click कर.

खाली **Run/Console** मध्ये असा output दिसायला हवा:

===== Account Details =====

Account Number : 1001

Account Holder : Reshma

Phone Number : 9876543210

Balance : ₹5000.0

 **इथे महत्वाचं काय शिकलो?**

```
Account account = new Account(...);
```

याचा अर्थ आपण Account class चा **object** तयार केला.

आणि:

```
account.getBalance()
```

यातून Account object मधला balance मिळवला.

म्हणजे आपला flow:

Account.java → Object तयार → Getter वापरला → Console मध्ये Output 

आता हा **code run करून console मध्ये काय output येतो ते मला पाठव. मग आपण पुढे Transaction.java बनवू.**

हो  हा error खूप common आहे. काळजी करू नकोस. याचा अर्थ तुझ्या Account.java मध्ये सध्या **no-argument constructor आहे/फक्त default constructor तयार झाला आहे**, पण Main.java मध्ये आपण 5 values देऊन object तयार करत आहोत.

आपण पटकन fix करूया.

1 Account.java पूर्णपणे replace कर

Account.java उघडून त्यातला सगळा code delete करून हा code paste कर:

```
package model;
```

```
public class Account {
```

```
private long accountNumber;

private String accountHolderName;

private String phoneNumber;

private int pin;

private double balance;


// Constructor

public Account(long accountNumber, String accountHolderName,
               String phoneNumber, int pin, double balance) {

    this.accountNumber = accountNumber;

    this.accountHolderName = accountHolderName;

    this.phoneNumber = phoneNumber;

    this.pin = pin;

    this.balance = balance;

}


// Getters

public long getAccountNumber() {

    return accountNumber;

}


public String getAccountHolderName() {

    return accountHolderName;

}


public String getPhoneNumber() {

    return phoneNumber;

}
```

```
public int getPin() {  
    return pin;  
}
```

```
public double getBalance() {  
    return balance;  
}
```

// Setters

```
public void setAccountHolderName(String accountHolderName) {  
    this.accountHolderName = accountHolderName;  
}
```

```
public void setPhoneNumber(String phoneNumber) {  
    this.phoneNumber = phoneNumber;  
}
```

```
public void setPin(int pin) {  
    this.pin = pin;  
}
```

```
public void setBalance(double balance) {  
    this.balance = balance;  
}  
}
```

2 Save कर

Ctrl + S दाब.

3 पुन्हा Main.java Run कर

Main.java वर right-click → **Run 'Main.main()'**

आता output:

===== Account Details =====

Account Number : 1001

Account Holder : Reshma

Phone Number : 9876543210

Balance : ₹5000.0

Error का आला?

आपल्या Main.java मध्ये:

```
new Account(1001, "Reshma", "9876543210", 1234, 5000);
```

असं 5 values दिल्या आहेत.

म्हणून Account class मध्ये हा constructor असणं आवश्यक आहे:

```
Account(long, String, String, int, double)
```

आधी तो constructor नसल्यामुळे compiler म्हणत होता:

required: no arguments

found: 5 arguments

म्हणजेच, “मला Account() हवा होता, पण तू 5 values देऊन Account(...) call केलास.”

आता Account.java replace करून Run कर. जर पुन्हा error आला तर पूर्ण error इथे paste कर, आपण त्याच वेळी fix करू.

हो! 🎉 एकदम perfect output आला आहे. तुझा पहिला Java program successfully run झाला. ✅

Output चा अर्थ

Account Number : 1001

Account Holder : Reshma

Phone Number : 9876543210

Balance : ₹5000.0

म्हणजे आपण:

1. Account class तयार केला ✅
2. त्यामध्ये variables ठेवले ✅
3. Constructor वापरून values दिल्या ✅
4. Account चा object तयार केला ✅

5. Getter methods वापरून values घेतल्या ✓

6. System.out.println() ने output दाखवला ✓

आणि शेवटी:

Process finished with exit code 0

याचा अर्थ **program कोणत्याही error शिवाय successfully complete झाला.** ¹⁰⁰

🧠 एक महत्वाची गोष्ट

आपण आता फक्त एक **account तयार करून त्याची माहिती display** केली आहे.

पुढे आपण actual banking functionality बनवणार आहोत:

Create Account



Deposit



Withdraw



Check Balance



Transaction History



Update Account



Delete Account

आता पुढचा step **Transaction.java** आहे. त्यातून आपण **transaction ID, Deposit/Withdraw type, amount आणि date/time** implement करू आणि त्यासोबत **LocalDateTime** देखील शिकू. 🚀

चला 👉 आता **Transaction.java** बनवूया.

Step 3 — Transaction.java

model package वर **Right Click → New → Java Class**

Class name:

Transaction

आता Transaction.java मध्ये हा code लिही:

```
package model;
```

```
import java.time.LocalDateTime;
```

```
public class Transaction {
```

```
    private int transactionId;
```

```
    private String type;
```

```
    private double amount;
```

```
    private LocalDateTime dateTime;
```

```
    // Constructor
```

```
    public Transaction(int transactionId, String type, double amount) {
```

```
        this.transactionId = transactionId;
```

```
        this.type = type;
```

```
        this.amount = amount;
```

```
        this.dateTime = LocalDateTime.now();
```

```
    }
```

```
    // Getters
```

```
    public int getTransactionId() {
```

```
        return transactionId;
```

```
    }
```

```
    public String getType() {
```

```
        return type;
```

```
    }
```

```
    public double getAmount() {
```

```
        return amount;
```

```
    }
```



```

public LocalDateTime getDateTime() {
    return dateTime;
}

// Display transaction details
public void displayTransaction() {
    System.out.println("Transaction ID : " + transactionId);
    System.out.println("Type      : " + type);
    System.out.println("Amount    : ₹" + amount);
    System.out.println("Date & Time  : " + dateTime);
}
}

```

🧠 थोडक्यात समजून घे

LocalDateTime

private LocalDateTime dateTime;

यात transaction कधी झाला ती date आणि time store होईल.

आणि:

this.dateTime = LocalDateTime.now();

याचा अर्थ transaction object तयार झाला त्यावेळची **current date आणि time** automatically store होईल.

उदा.:

Transaction ID : 101

Type : Deposit

Amount : ₹2000.0

Date & Time : 2026-09-01T17:20:35

⚠️ आता Run करू नकोस

Transaction.java मध्ये main() method नाही, त्यामुळे direct run करण्याची गरज नाही.

फक्त file तयार करून code paste कर. झाल्यावर मला “Transaction.java तयार झाला” असं सांग. मग आपण Main.java मधून त्याची test करू.

छान!  आता Transaction.java योग्य आहे का ते Main.java मधून test करूया.

तुझ्या सध्याच्या Main.java मध्ये account code आहे. त्याच्या खाली transaction चा हा भाग add कर:

```
import model.Account;
```

```
import model.Transaction;
```

```
public class Main {
```

```
    public static void main(String[] args) {
```

```
        Account account = new Account(
```

```
            1001,
```

```
            "Reshma",
```

```
            "9876543210",
```

```
            1234,
```

```
            5000
```

```
        );
```

```
        System.out.println("===== Account Details =====");
```

```
        System.out.println("Account Number : " + account.getAccountNumber());
```

```
        System.out.println("Account Holder : " + account.getAccountHolderName());
```

```
        System.out.println("Phone Number : " + account.getPhoneNumber());
```

```
        System.out.println("Balance : ₹" + account.getBalance());
```

```
        // Create Transaction
```

```
        Transaction transaction = new Transaction(
```

```
            101,
```

```
            "Deposit",
```

```
            2000
```

```

);

System.out.println("\n===== Transaction Details =====");

transaction.displayTransaction();

}

}

```

▶ आता Run कर

Main.java → Right Click → Run 'Main.main()'

Output साधारण असा येईल:

===== Account Details =====

Account Number : 1001

Account Holder : Reshma

Phone Number : 9876543210

Balance : ₹5000.0

===== Transaction Details =====

Transaction ID : 101

Type : Deposit

Amount : ₹2000.0

Date & Time : 2026-09-01T...

Date & Time तुझ्या laptop च्या current time प्रमाणे वेगळा असेल, ते normal आहे. 👍

🧠 इथे एक नवीन concept

Transaction transaction = new Transaction(101, "Deposit", 2000);

इथे आपण Transaction class चा **object** तयार केला.

आणि:

transaction.displayTransaction();

यामुळे त्या transaction ची पूर्ण माहिती display झाली.

Run करून आलेला output मला पाठव. मग आपण पुढचा महत्वाचा भाग – BankService.java आणि Deposit/Withdraw functionality — सुरू करू. 🚀

हो 👉 हा error पाहता Main.java मध्ये code चुकीच्या ठिकाणी paste झाला असण्याची शक्यता जास्त आहे. काळजी करू नकोस.

आपण सर्वात सोपा मार्ग घेऊया: पूर्ण Main.java replace करूया.

1. Main.java उघड

त्यातला सगळा code delete कर.

आणि खालील पूर्ण code paste कर:

```
import model.Account;
```

```
import model.Transaction;
```

```
public class Main {
```

```
    public static void main(String[] args) {
```

```
        Account account = new Account(
```

```
            1001,
```

```
            "Reshma",
```

```
            "9876543210",
```

```
            1234,
```

```
            5000
```

```
        );
```

```
        System.out.println("==== Account Details =====");
```

```
        System.out.println("Account Number : " + account.getAccountNumber());
```

```
        System.out.println("Account Holder : " + account.getAccountHolderName());
```

```
        System.out.println("Phone Number : " + account.getPhoneNumber());
```

```
        System.out.println("Balance : ₹" + account.getBalance());
```

```
        // Create Transaction
```

```
        Transaction transaction = new Transaction(
```

```
            101,
```

```

        "Deposit",
        2000
    );

    System.out.println("\n===== Transaction Details =====");
    transaction.displayTransaction();
}
}

```

2. Save कर

Ctrl + S

3. Run कर

Main.java वर Right Click → **Run 'Main.main()'**

Expected output:

===== Account Details =====

Account Number : 1001

Account Holder : Reshma

Phone Number : 9876543210

Balance : ₹5000.0

===== Transaction Details =====

Transaction ID : 101

Type : Deposit

Amount : ₹2000.0

Date & Time : 2026-09-01T...

! Error का आला?

class, interface, enum, or record expected हा error साधारणपणे Java ला class च्या बाहेर काही code सापडल्यावर येतो.

उदा. असा code चुकतो:

```

public class Main {
    // code

```

```
}
```

System.out.println("Hello"); // ❌ class च्या बाहेर

म्हणूनच पूर्ण Main.java replace करणं safest आहे.

आता Run करून जो output/error येतो तो मला पाठव. मग पुढे जाऊ.

हो 👍 Main.java मधला आधीचा Account चा code delete करायचा, पण Account.java मधला code अजिबात delete करू नकोस.

फरक लक्षात ठेव:

src

|

|— Main.java ← हा पूर्ण replace करायचा ✅

|

└─ model

 |— Account.java ← हा तसाच ठेवायचा ❌ delete करू नको

 └─ Transaction.java ← हा तसाच ठेवायचा ❌ delete करू नको

म्हणजे:

- **Account.java** → आधीचा पूर्ण code तसाच राहू दे.
- **Transaction.java** → आताच तयार केलेला code तसाच राहू दे.
- **Main.java** → आधीचा code पूर्ण delete करून मी दिलेला नवीन code paste कर.

नंतर Run → Main.main() कर.

जर output आला तर आपण पुढे **Deposit आणि Withdraw** functionality बनवू. 🚀

हो 👍 हा error अगदी स्पष्ट आहे: **Account class चा code चुकीने Main.java मध्ये गेला आहे** किंवा file names/code mismatch झाले आहेत.

आपण एकदम व्यवस्थित fix करूया.

1 Project structure तपास

IntelliJ च्या डाव्या बाजूला हे असायला हवं:

BankingManagementSystem

```
└─ src
    └─ Main.java
    |
    └─ model
        └─ Account.java
        └─ Transaction.java
```

2 Account.java

model → Account.java उघड़.

त्यात **फक्त Account चा code** असावा:

```
package model;
```

```
public class Account {
```

```
    private long accountNumber;
```

```
    private String accountHolderName;
```

```
    private String phoneNumber;
```

```
    private int pin;
```

```
    private double balance;
```

```
    public Account(long accountNumber, String accountHolderName,
                   String phoneNumber, int pin, double balance) {
```

```
        this.accountNumber = accountNumber;
```

```
        this.accountHolderName = accountHolderName;
```

```
        this.phoneNumber = phoneNumber;
```

```
        this.pin = pin;
```

```
        this.balance = balance;
```

```
    }
```

```
    public long getAccountNumber() {
```

```
        return accountNumber;
    }
}
```

```
public String getAccountHolderName() {
    return accountHolderName;
}
```

```
public String getPhoneNumber() {
    return phoneNumber;
}
```

```
public int getPin() {
    return pin;
}
```

```
public double getBalance() {
    return balance;
}
```

```
public void setAccountHolderName(String accountHolderName) {
    this.accountHolderName = accountHolderName;
}
```

```
public void setPhoneNumber(String phoneNumber) {
    this.phoneNumber = phoneNumber;
}
```

```
public void setPin(int pin) {
    this.pin = pin;
}
```



```
    public void setBalance(double balance) {  
        this.balance = balance;  
    }  
}
```

3 Transaction.java

model → Transaction.java मध्ये फक्त Transaction चा code असावा.

```
package model;
```

```
import java.time.LocalDateTime;
```

```
public class Transaction {
```

```
    private int transactionId;
```

```
    private String type;
```

```
    private double amount;
```

```
    private LocalDateTime dateTime;
```

```
    public Transaction(int transactionId, String type, double amount) {
```

```
        this.transactionId = transactionId;
```

```
        this.type = type;
```

```
        this.amount = amount;
```

```
        this.dateTime = LocalDateTime.now();
```

```
    }
```

```
    public int getTransactionId() {
```

```
        return transactionId;
```

```
    }
```

```
    public String getType() {
```

```
        return type;
```

```
    }
```

```
public double getAmount() {  
    return amount;  
}
```

```
public LocalDateTime getDateTime() {  
    return dateTime;  
}
```

```
public void displayTransaction() {  
    System.out.println("Transaction ID : " + transactionId);  
    System.out.println("Type      : " + type);  
    System.out.println("Amount    : ₹" + amount);  
    System.out.println("Date & Time  : " + dateTime);  
}  
}
```

Main.java

आणि फक्त Main.java मध्ये हा code असावा:

```
import model.Account;
```

```
import model.Transaction;
```

```
public class Main {
```

```
    public static void main(String[] args) {
```

```
        Account account = new Account(  
            1001,  
            "Reshma",  
            "9876543210",  
            1234,  
            5000
```

```
);
```

```
System.out.println("==== Account Details =====");
```

```
System.out.println("Account Number : " + account.getAccountNumber());
```

```
System.out.println("Account Holder : " + account.getAccountHolderName());
```

```
System.out.println("Phone Number : " + account.getPhoneNumber());
```

```
System.out.println("Balance : ₹" + account.getBalance());
```

```
Transaction transaction = new Transaction(
```

```
    101,
```

```
    "Deposit",
```

```
    2000
```

```
);
```

```
System.out.println("\n==== Transaction Details =====");
```

```
transaction.displayTransaction();
```

```
}
```

```
}
```

● सर्वात महत्वाची गोष्ट

public class Account → **Account.java** मध्येच.

public class Transaction → **Transaction.java** मध्येच.

public class Main → **Main.java** मध्येच.

एक file मध्ये दुसऱ्या class चा पूर्ण code paste करू नकोस.

आता हे तीन files check कर आणि Run कर. जर पुन्हा error आला, तर error चा screenshot पाठव; मी नेमका कोणता file चुकीचा आहे ते सांगतो.

एकदम मस्त! 🎉 आता दोन्ही classes successfully काम करत आहेत. ✅

तुझा output पूर्णपणे correct आहे:

```
===== Account Details =====
```

```
Account Number : 1001
```

Account Holder : Reshma

Phone Number : 9876543210

Balance : ₹5000.0

===== Transaction Details =====

Transaction ID : 101

Type : Deposit

Amount : ₹2000.0

Date & Time : 2026-09-01T17:41:01...

आणि:

Process finished with exit code 0

याचा अर्थ कोणताही error नाही. **100**

आतापर्यंत आपण काय केलं?

Account.java

↓

Account Object

↓

Account Details

आणि:

Transaction.java

↓

Transaction Object

↓

Deposit Transaction

↓

Date & Time

यातून तू practically हे concepts वापरलेस:

- ☒ Class & Object
- ☒ Constructor
- ☒ Encapsulation

-  Getters/Setters
-  LocalDateTime
-  import आणि package
-  Multiple classes
-  main() method

आता पुढचा important step

आता आपण **BankService.java** तयार करू.

त्यात actual banking operations येतील:

BankService

|

└─ createAccount()

└─ deposit()

└─ withdraw()

└─ checkBalance()

└─ searchAccount()

└─ updateAccount()

└─ deleteAccount()

म्हणजे आतापर्यंत आपण **data structure तयार केला** आहे; आता त्यावर **actual banking logic** लिहायला सुरुवात करू.

पुढचा step: BankService.java + ArrayList<Account> — इथे तुला **Collections (ArrayList)** हा Core Java मधला खूप important concept practically समजेल. 

छान  आता आपण project ला **actual banking functionality** देणार आहोत.

Step 4 — BankService.java

आधी model package मध्ये Account आणि Transaction आहेत. आता src वर right-click करून:

New → Package → service

नंतर service वर right-click:

New → Java Class → BankService

BankService.java मध्ये हा code टाक:

```
package service;

import model.Account;

import java.util.ArrayList;

public class BankService {

    private ArrayList<Account> accounts = new ArrayList<>();

    // Create Account
    public void createAccount(Account account) {
        accounts.add(account);
        System.out.println("Account created successfully!");
    }

    // Search Account
    public Account searchAccount(long accountNumber) {

        for (Account account : accounts) {

            if (account.getAccountNumber() == accountNumber) {
                return account;
            }
        }

        return null;
    }

    // Display All Accounts
    public void displayAllAccounts() {
```

```

if (accounts.isEmpty()) {
    System.out.println("No accounts found.");
    return;
}

for (Account account : accounts) {

    System.out.println("-----");
    System.out.println("Account Number : " + account.getAccountNumber());
    System.out.println("Account Holder : " + account.getAccountHolderName());
    System.out.println("Phone Number : " + account.getPhoneNumber());
    System.out.println("Balance      : ₹" + account.getBalance());
}
}
}

```

आता नवीन concept — ArrayList

ही line:

```
private ArrayList<Account> accounts = new ArrayList<>();
```

याचा अर्थ आपण **अनेक Account objects एकाच ठिकाणी store** करू शकतो.

उदा.:

accounts

|

└─ Account 1001 - Reshma

└─ Account 1002 - Rahul

└─ Account 1003 - Priya

└─ Account 1004 - Amit

आणि:

```
accounts.add(account);
```

याचा अर्थ नवीन account ArrayList मध्ये add करणे.

तर:

```
searchAccount(1001);
```

केल्यावर त्या account चा object आपल्याला मिळेल.

⚠ **आता BankService.java Run करू नकोस**

यात main() method नाही.

फक्त file तयार करून code paste कर आणि Save कर.

झाल्यावर मला “BankService तयार झाला” असं सांग. मग आपण Main.java मधून **Create Account + Search Account + Display All Accounts** test करू. 🚀

छान! 🔥 आता BankService ला आपल्या Main.java सोबत connect करूया आणि **ArrayList मध्ये accounts add/search** करून पाहूया.

Step 5 — Main.java update कर

Main.java मधला सगळा code delete करून हा code paste कर:

```
import model.Account;
```

```
import model.Transaction;
```

```
import service.BankService;
```

```
public class Main {
```

```
    public static void main(String[] args) {
```

```
        // Create BankService object
```

```
        BankService bankService = new BankService();
```

```
        // Create Account 1
```

```
        Account account1 = new Account(
```

```
            1001,
```

```
            "Reshma",
```



```
        "9876543210",
        1234,
        5000
    );

    // Create Account 2
    Account account2 = new Account(
        1002,
        "Rahul",
        "9876543211",
        5678,
        10000
    );

    // Add accounts
    bankService.createAccount(account1);
    bankService.createAccount(account2);

    // Display all accounts
    System.out.println("\n===== All Accounts =====");
    bankService.displayAllAccounts();

    // Search account
    System.out.println("\n===== Search Account =====");

    Account foundAccount = bankService.searchAccount(1001);

    if (foundAccount != null) {
        System.out.println("Account Found!");
        System.out.println("Account Holder : "
            + foundAccount.getAccountHolderName());
    }
}
```

```

        System.out.println("Balance : ₹"
            + foundAccount.getBalance());
    } else {
        System.out.println("Account not found.");
    }
}
}
}

```

 **Run कर**

Main.java → Right Click → **Run 'Main.main()'**

तुला साधारण असा output मिळेल:

Account created successfully!

Account created successfully!

===== All Accounts =====

Account Number : 1001

Account Holder : Reshma

Phone Number : 9876543210

Balance : ₹5000.0

Account Number : 1002

Account Holder : Rahul

Phone Number : 9876543211

Balance : ₹10000.0

===== Search Account =====

Account Found!

Account Holder : Reshma

Balance : ₹5000.0

 **इथे एक महत्वाची गोष्ट**

आपण आता:

```
BankService bankService = new BankService();
```

यातून BankService चा object तयार केला.

नंतर:

```
bankService.createAccount(account1);
```

```
bankService.createAccount(account2);
```

यातून दोन accounts ArrayList मध्ये गेले.

आणि:

```
Account foundAccount = bankService.searchAccount(1001);
```

यातून account number **1001** शोधला.

म्हणजे आता आपला project फक्त data print करत नाहीये — **actual application logic तयार व्हायला सुरुवात झाली आहे.** 🙌

हा code run करून output मला पाठव. मग पुढे आपण सर्वात important functionality — **Deposit आणि Withdraw** — बनवू.

एकदम perfect! 🎉🔥 **Output 100% correct आहे.**

आता तुझ्या project मध्ये हे properly काम करत आहे:

- ☒ Account class
- ☒ Transaction class
- ☒ BankService class
- ☒ ArrayList<Account>
- ☒ Multiple accounts
- ☒ Create Account
- ☒ Search Account
- ☒ Display All Accounts
- ☒ Constructor
- ☒ Encapsulation
- ☒ Getter/Setter
- ☒ LocalDateTime

-  Packages & Imports

आता पुढचा level 💰

आता आपण **Deposit आणि Withdraw** functionality बनवू.

उदा.:

Current Balance = ₹5000

Deposit ₹2000

↓

New Balance = ₹7000

Withdraw ₹1500

↓

New Balance = ₹5500

आणि इथे आपण **Exception Handling** देखील वापरणार आहोत.

उदा. balance ₹5000 असेल आणि user ने ₹6000 withdraw करण्याचा प्रयत्न केला तर:

Insufficient Balance!

हे आपल्या **Core Java project मधलं खूप important real-world logic** असेल.

पुढच्या step मध्ये BankService.java मध्ये deposit() आणि withdraw() methods add करूया. 🚀

चला 👍 आता **Deposit + Withdraw** implement करूया. यामुळे project खरंच banking system सारखा काम करायला लागेल.

Step 6 — BankService.java update

service → BankService.java उघड.

आताचा code पूर्ण delete करून हा updated code paste कर:

```
package service;
```

```
import model.Account;
```

```
import java.util.ArrayList;
```

```
public class BankService {

    private ArrayList<Account> accounts = new ArrayList<>();

    // Create Account
    public void createAccount(Account account) {
        accounts.add(account);
        System.out.println("Account created successfully!");
    }

    // Search Account
    public Account searchAccount(long accountNumber) {

        for (Account account : accounts) {

            if (account.getAccountNumber() == accountNumber) {
                return account;
            }
        }

        return null;
    }

    // Display All Accounts
    public void displayAllAccounts() {

        if (accounts.isEmpty()) {
            System.out.println("No accounts found.");
            return;
        }
    }
}
```

```
for (Account account : accounts) {

    System.out.println("-----");
    System.out.println("Account Number : " + account.getAccountNumber());
    System.out.println("Account Holder : " + account.getAccountHolderName());
    System.out.println("Phone Number : " + account.getPhoneNumber());
    System.out.println("Balance : ₹" + account.getBalance());
}
}
```

```
// Deposit Money
```

```
public void deposit(long accountNumber, double amount) {
```

```
    Account account = searchAccount(accountNumber);
```

```
    if (account == null) {
```

```
        System.out.println("Account not found.");
```

```
        return;
```

```
    }
```

```
    if (amount <= 0) {
```

```
        System.out.println("Invalid deposit amount.");
```

```
        return;
```

```
    }
```

```
    double newBalance = account.getBalance() + amount;
```

```
    account.setBalance(newBalance);
```

```
    System.out.println("Deposit successful!");
```

```
    System.out.println("Deposited Amount : ₹" + amount);
```

```
        System.out.println("New Balance    : ₹" + account.getBalance());  
    }
```

```
// Withdraw Money
```

```
public void withdraw(long accountNumber, double amount) {
```

```
    Account account = searchAccount(accountNumber);
```

```
    if (account == null) {
```

```
        System.out.println("Account not found.");
```

```
        return;
```

```
    }
```

```
    if (amount <= 0) {
```

```
        System.out.println("Invalid withdrawal amount.");
```

```
        return;
```

```
    }
```

```
    if (amount > account.getBalance()) {
```

```
        System.out.println("Insufficient balance!");
```

```
        return;
```

```
    }
```

```
    double newBalance = account.getBalance() - amount;
```

```
    account.setBalance(newBalance);
```

```
    System.out.println("Withdrawal successful!");
```

```
    System.out.println("Withdrawn Amount : ₹" + amount);
```

```
    System.out.println("New Balance    : ₹" + account.getBalance());
```

```
}
```

```
}
```

Deposit कसं काम करतं?

आपण:

```
double newBalance = account.getBalance() + amount;
```

उदा.:

Balance = ₹5000

Deposit = ₹2000

$5000 + 2000 = ₹7000$

मग:

```
account.setBalance(newBalance);
```

मुळे Account चा balance update होतो.

Withdraw

```
if (amount > account.getBalance()) {  
    System.out.println("Insufficient balance!");  
    return;  
}
```

Balance पेक्षा जास्त पैसे withdraw केले तर transaction थांबतो.

Step 7 — Main.java मध्ये test कर

आता Main.java मधला code पूर्ण बदलण्याची गरज नाही. `bankService.createAccount(account2);` नंतर हा code add कर:

```
// Deposit
```

```
System.out.println("\n===== Deposit =====");
```

```
bankService.deposit(1001, 2000);
```

```
// Withdraw
```

```
System.out.println("\n===== Withdraw =====");
```

```
bankService.withdraw(1001, 1500);
```



```
// Check Updated Balance
System.out.println("\n===== Updated Balance =====");
Account updatedAccount = bankService.searchAccount(1001);

System.out.println("Account Holder : "
    + updatedAccount.getAccountHolderName());

System.out.println("Current Balance : ₹"
    + updatedAccount.getBalance());
```

Expected output

===== Deposit =====

Deposit successful!

Deposited Amount : ₹2000.0

New Balance : ₹7000.0

===== Withdraw =====

Withdrawal successful!

Withdrawn Amount : ₹1500.0

New Balance : ₹5500.0

===== Updated Balance =====

Account Holder : Reshma

Current Balance : ₹5500.0

हे run करून output पाठव. मग आपण deliberately ₹6000 withdraw करून insufficient balance condition test करू — त्यानंतर Exception Handling properly add करू. 🚀

एकदम correct! 🔥 तुझा Deposit + Withdraw logic perfectly काम करतोय. ✅

तुझ्या output मध्ये:

₹5000

+ ₹2000 Deposit

= ₹7000

₹7000

- ₹1500 Withdraw

= ₹5500

आणि नंतर All Accounts मध्येही updated balance ₹5500 दिसतोय. म्हणजे setBalance() नीट काम करत आहे. **100**

आतापर्यंत Project Progress

Account



Transaction



BankService



ArrayList



Create Account



Search Account



Deposit



Withdraw



Check Updated Balance

 आता एक important Core Java concept

आता आपण **Exception Handling** शिकू.

आता चुकीचा amount दिल्यावर आपण:

```
if (amount > account.getBalance()) {  
    System.out.println("Insufficient balance!");  
}
```

```
return;  
}
```

असं करत आहोत.

पण professional Java application मध्ये आपण custom exception तयार करू शकतो:

InsufficientBalanceException

आणि मग:

Withdraw ₹6000

↓

Balance ₹5500

↓

InsufficientBalanceException

↓

"Insufficient Balance!"

यातून तुला **try-catch, throw, throws, custom exception** हे Core Java concepts practically समजतील.

पुढचा step: InsufficientBalanceException.java तयार करूया. 🚀

छान 👍 आता आपण **Custom Exception** बनवूया. हा Core Java project मधला important concept आहे.

Step 8 — InsufficientBalanceException.java

1. exception package तयार कर

IntelliJ मध्ये:

src → Right Click → New → Package

Package name:

exception

2. Exception class तयार कर

exception package वर:

Right Click → New → Java Class

Class name:

InsufficientBalanceException

3. हा code paste कर

```
package exception;
```

```
public class InsufficientBalanceException extends Exception {
```

```
    public InsufficientBalanceException(String message) {
```

```
        super(message);
```

```
    }
```

```
}
```

🧠 हे काय आहे?

extends Exception

याचा अर्थ आपली InsufficientBalanceException ही Java च्या Exception class वर आधारित आहे.

आणि:

```
super(message);
```

आपण exception ला message देतो.

उदा.:

Insufficient Balance! Available balance: ₹5500.0

पुढे काय करणार?

आता BankService मध्ये withdraw() method अशी बनवू की insufficient balance झाल्यावर आपली **custom exception throw** होईल.

मग Main.java मध्ये:

```
try
```

```
↓
```

```
withdraw()
```

```
↓
```

Exception आली?

```
↓
```

```
catch
```

```
↓
```

Error message

असं **try-catch + throw + throws** practically वापरू.

आता फक्त `InsufficientBalanceException.java` तयार करून **code paste + save** कर.

झाल्यावर “तयार झाला” असं सांग.

छान 👍 आता आपण त्या custom exception ला आपल्या **withdraw functionality** मध्ये वापरूया.

Step 9 — `BankService.java` update

service → `BankService.java` उघड आणि पूर्ण code हा ठेव:

```
package service;
```

```
import exception.InsufficientBalanceException;
```

```
import model.Account;
```

```
import java.util.ArrayList;
```

```
public class BankService {
```

```
    private ArrayList<Account> accounts = new ArrayList<>();
```

```
    // Create Account
```

```
    public void createAccount(Account account) {
```

```
        accounts.add(account);
```

```
        System.out.println("Account created successfully!");
```

```
    }
```

```
    // Search Account
```

```
    public Account searchAccount(long accountNumber) {
```

```
        for (Account account : accounts) {
```

```
            if (account.getAccountNumber() == accountNumber) {
```

```

        return account;
    }
}

return null;
}

// Display All Accounts
public void displayAllAccounts() {

    if (accounts.isEmpty()) {
        System.out.println("No accounts found.");
        return;
    }

    for (Account account : accounts) {

        System.out.println("-----");
        System.out.println("Account Number : " + account.getAccountNumber());
        System.out.println("Account Holder : " + account.getAccountHolderName());
        System.out.println("Phone Number : " + account.getPhoneNumber());
        System.out.println("Balance      : ₹" + account.getBalance());
    }
}

// Deposit Money
public void deposit(long accountNumber, double amount) {

    Account account = searchAccount(accountNumber);

    if (account == null) {

```

```
        System.out.println("Account not found.");
        return;
    }
```

```
    if (amount <= 0) {
        System.out.println("Invalid deposit amount.");
        return;
    }
```

```
    double newBalance = account.getBalance() + amount;
    account.setBalance(newBalance);
```

```
    System.out.println("Deposit successful!");
    System.out.println("Deposited Amount : ₹" + amount);
    System.out.println("New Balance    : ₹" + account.getBalance());
}
```

```
// Withdraw Money
```

```
public void withdraw(long accountNumber, double amount)
    throws InsufficientBalanceException {
```

```
    Account account = searchAccount(accountNumber);
```

```
    if (account == null) {
        System.out.println("Account not found.");
        return;
    }
```

```
    if (amount <= 0) {
        System.out.println("Invalid withdrawal amount.");
        return;
    }
```

```

    }

    if (amount > account.getBalance()) {
        throw new InsufficientBalanceException(
            "Insufficient Balance! Available balance: ₹"
            + account.getBalance()
        );
    }

    double newBalance = account.getBalance() - amount;
    account.setBalance(newBalance);

    System.out.println("Withdrawal successful!");
    System.out.println("Withdrawn Amount : ₹" + amount);
    System.out.println("New Balance    : ₹" + account.getBalance());
}
}

```

🔑 नवीन काय केलं?

आपल्या method मध्ये:

throws InsufficientBalanceException

म्हणजे ही method exception throw करू शकते.

आणि:

throw new InsufficientBalanceException(...);

Balance कमी असेल तर आपण **स्वतःची custom exception throw** करत आहोत.

Step 10 — Main.java मध्ये try-catch

आता Main.java मध्ये withdraw() call असा बदल.

जुना:

```
bankService.withdraw(1001, 1500);
```


delete करून:

```
try {  
    bankService.withdraw(1001, 1500);  
} catch (InsufficientBalanceException e) {  
    System.out.println(e.getMessage());  
}
```

आणि Main.java च्या वर import add कर:

```
import exception.InsufficientBalanceException;
```

म्हणून imports असे दिसतील:

```
import exception.InsufficientBalanceException;
```

```
import model.Account;
```

```
import model.Transaction;
```

```
import service.BankService;
```

 आता test करूया

पहिल्यांदा:

```
bankService.withdraw(1001, 1500);
```

याने normal withdrawal होईल.

नंतर दुसरा test कर:

```
try {  
    bankService.withdraw(1001, 6000);  
} catch (InsufficientBalanceException e) {  
    System.out.println(e.getMessage());  
}
```

आपल्याकडे balance ₹5500 असेल, त्यामुळे output:

Insufficient Balance! Available balance: ₹5500.0

आणि program crash होणार नाही. 

हे run करून output पाठव. त्यानंतर आपण Transaction History (ArrayList<Transaction>) जोडू-म्हणजे प्रत्येक Deposit/Withdraw ची history store होईल. 

हो 👍 Step 10 मध्ये confusion होणं साहजिक आहे. काय delete करायचं आणि काय paste करायचं ते exact सांगतो.

1 Main.java उघड

सध्या तुझ्या Main.java मध्ये import च्या वर/खाली हे असतील:

```
import model.Account;
import model.Transaction;
import service.BankService;
```

त्यामध्ये हा एक import add कर:

```
import exception.InsufficientBalanceException;
```

म्हणजे सुरुवातीला असे दिसले पाहिजे:

```
import exception.InsufficientBalanceException;
import model.Account;
import model.Transaction;
import service.BankService;
```

2 आता withdraw चा जुना भाग शोध

तुझ्या Main.java मध्ये हा भाग आहे:

```
System.out.println("\n===== Withdraw =====");
bankService.withdraw(1001, 1500);
```

फक्त ही एक line:

```
bankService.withdraw(1001, 1500);
```

✗ Delete कर.

System.out.println("\n===== Withdraw ====="); delete करू नकोस.

3 त्या deleted line च्या जागी हे paste कर:

```
try {
    bankService.withdraw(1001, 1500);
} catch (InsufficientBalanceException e) {
```

```
System.out.println(e.getMessage());  
}
```

म्हणजे पूर्ण section असा दिसला पाहिजे:

```
System.out.println("\n===== Withdraw =====");
```

```
try {  
    bankService.withdraw(1001, 1500);  
} catch (InsufficientBalanceException e) {  
    System.out.println(e.getMessage());  
}
```

आता दुसरा test add करू

त्याच्या लगेच खाली हे paste कर:

```
System.out.println("\n===== Insufficient Balance Test =====");
```

```
try {  
    bankService.withdraw(1001, 6000);  
} catch (InsufficientBalanceException e) {  
    System.out.println(e.getMessage());  
}
```

कारण आपल्या account मध्ये ₹5500 आहेत आणि आपण ₹6000 withdraw करणार आहोत.

5 शेवटी Main.java साधारण असा दिसेल

```
import exception.InsufficientBalanceException;  
  
import model.Account;  
  
import model.Transaction;  
  
import service.BankService;  
  
  
public class Main {
```

```
public static void main(String[] args) {

    BankService bankService = new BankService();

    Account account1 = new Account(
        1001,
        "Reshma",
        "9876543210",
        1234,
        5000
    );

    Account account2 = new Account(
        1002,
        "Rahul",
        "9876543211",
        5678,
        10000
    );

    bankService.createAccount(account1);
    bankService.createAccount(account2);

    System.out.println("\n===== Deposit =====");
    bankService.deposit(1001, 2000);

    System.out.println("\n===== Withdraw =====");

    try {
        bankService.withdraw(1001, 1500);
    } catch (InsufficientBalanceException e) {
```

```

        System.out.println(e.getMessage());
    }

    System.out.println("\n===== Insufficient Balance Test =====");

    try {
        bankService.withdraw(1001, 6000);
    } catch (InsufficientBalanceException e) {
        System.out.println(e.getMessage());
    }

    System.out.println("\n===== Updated Balance =====");

    Account updatedAccount = bankService.searchAccount(1001);

    System.out.println("Account Holder : "
        + updatedAccount.getAccountHolderName());

    System.out.println("Current Balance : ₹"
        + updatedAccount.getBalance());

    System.out.println("\n===== All Accounts =====");
    bankService.displayAllAccounts();
}
}

```

आता Transaction चा code Main मधून काढला आहे, कारण तो आपण पुढच्या step मध्ये proper transaction history साठी वापरणार आहोत.

आता **Save → Run** कर. Output मध्ये शेवटी:

===== Insufficient Balance Test =====

Insufficient Balance! Available balance: ₹5500.0

असं आलं तर आपलं Custom Exception successfully काम करतंय. ✓

एकदम perfect! 🎉🔥 Custom Exception सुद्धा successfully काम करत आहे.

तुझ्या output मध्ये हे सगळं योग्य आहे:

Deposit ₹2000

₹5000 → ₹7000

Withdraw ₹1500

₹7000 → ₹5500

Withdraw ₹6000

→ Insufficient Balance!

आणि महत्वाचं म्हणजे program crash झाला नाही. catch ने exception handle केली. ✓

🧠 आता तुला हे concepts practically आले

- try
- catch
- throw
- throws
- Custom Exception
- ArrayList
- Class/Object
- Constructor
- Encapsulation
- Getter/Setter
- LocalDateTime

🚀 पुढचा Step: Transaction History

आता आपल्या project मध्ये एक मोठी कमी आहे:

आपण Deposit/Withdraw करतोय, पण त्याची history कुठेही save होत नाही.

आपल्याला असं पाहिजे:

===== Transaction History =====

Transaction ID : 1

Type : Deposit

Amount : ₹2000

Date & Time : ...

Transaction ID : 2

Type : Withdraw

Amount : ₹1500

Date & Time : ...

यासाठी आपण:

ArrayList<Transaction>

वापरणार आहोत.

म्हणजे:

Account



BankService



Deposit / Withdraw



Transaction तयार



Transaction History मध्ये save

हा भाग केल्यावर आपला project **जास्त real banking application सारखा** वाटायला लागेल. 🙌

पुढच्या step मध्ये आपण BankService मध्ये **Transaction History जोडू**.

छान 🙌 आता आपण **Transaction History** जोडूया. या step मध्ये Transaction.java आधीच तयार आहे, त्यामुळे फक्त BankService.java update करायचा आहे.

Step 11 — Transaction History

BankService.java उघड़

सगळा जुना code delete करून हा पूर्ण code paste कर:

```
package service;
```

```
import exception.InsufficientBalanceException;
```

```
import model.Account;
```

```
import model.Transaction;
```

```
import java.util.ArrayList;
```

```
public class BankService {
```

```
    private ArrayList<Account> accounts = new ArrayList<>();
```

```
    private ArrayList<Transaction> transactions = new ArrayList<>();
```

```
    private int transactionCounter = 1;
```

```
    // Create Account
```

```
    public void createAccount(Account account) {
```

```
        accounts.add(account);
```

```
        System.out.println("Account created successfully!");
```

```
    }
```

```
    // Search Account
```

```
    public Account searchAccount(long accountNumber) {
```



```
for (Account account : accounts) {

    if (account.getAccountNumber() == accountNumber) {
        return account;
    }
}

return null;
}
```

// Display All Accounts

```
public void displayAllAccounts() {

    if (accounts.isEmpty()) {
        System.out.println("No accounts found.");
        return;
    }

    for (Account account : accounts) {
```

```
        System.out.println("-----");
```

```
        System.out.println("Account Number : "
            + account.getAccountNumber());
```

```
        System.out.println("Account Holder : "
            + account.getAccountHolderName());
```

```
        System.out.println("Phone Number : "
            + account.getPhoneNumber());
```

```
        System.out.println("Balance    : ₹"
            + account.getBalance());
    }
}
```

// Deposit Money

```
public void deposit(long accountNumber, double amount) {
```

```
    Account account = searchAccount(accountNumber);
```

```
    if (account == null) {
```

```
        System.out.println("Account not found.");
```

```
        return;
```

```
    }
```

```
    if (amount <= 0) {
```

```
        System.out.println("Invalid deposit amount.");
```

```
        return;
```

```
    }
```

```
    double newBalance =
```

```
        account.getBalance() + amount;
```

```
    account.setBalance(newBalance);
```

// Create Deposit Transaction

```
Transaction transaction =
```

```
    new Transaction(
```

```

        transactionCounter++,

        "Deposit",

        amount

    );

    transactions.add(transaction);


    System.out.println("Deposit successful!");
    System.out.println("Deposited Amount : ₹" + amount);
    System.out.println("New Balance    : ₹"
        + account.getBalance());
}


// Withdraw Money
public void withdraw(long accountNumber, double amount)
    throws InsufficientBalanceException {

    Account account = searchAccount(accountNumber);

    if (account == null) {
        System.out.println("Account not found.");
        return;
    }

    if (amount <= 0) {
        System.out.println("Invalid withdrawal amount.");
        return;
    }

```

```

if (amount > account.getBalance()) {

    throw new InsufficientBalanceException(
        "Insufficient Balance! Available balance: ₹"
        + account.getBalance()
    );
}

double newBalance =
    account.getBalance() - amount;

account.setBalance(newBalance);

// Create Withdrawal Transaction
Transaction transaction =
    new Transaction(
        transactionCounter++,
        "Withdraw",
        amount
    );

transactions.add(transaction);

System.out.println("Withdrawal successful!");
System.out.println("Withdrawn Amount : ₹" + amount);
System.out.println("New Balance    : ₹"
    + account.getBalance());
}

```

```
// Display Transaction History
public void displayTransactionHistory() {

    if (transactions.isEmpty()) {

        System.out.println("No transactions found.");

        return;
    }

    for (Transaction transaction : transactions) {

        System.out.println("-----");

        transaction.displayTransaction();
    }
}
}
```

🧠 नवीन काय केलं?

ही line:

```
private ArrayList<Transaction> transactions = new ArrayList<>();
```

आपली **Transaction History** आहे.

Deposit successful झाल्यावर:

```
transactions.add(transaction);
```

मुळे transaction history मध्ये save होते.

Withdraw successful झाल्यावरही transaction save होते.

आणि:

```
transactionCounter++
```

मुळे transaction IDs automatically:

1

2

3

4

...

अशा वाढत जातील.

Step 12 — Main.java मध्ये History display कर

Main.java मध्ये शेवटी, displayAllAccounts() नंतर हे add कर:

```
System.out.println("\n===== Transaction History =====");
```

```
bankService.displayTransactionHistory();
```

म्हणजे शेवटचा भाग असा दिसेल:

```
System.out.println("\n===== All Accounts =====");
```

```
bankService.displayAllAccounts();
```

```
System.out.println("\n===== Transaction History =====");
```

```
bankService.displayTransactionHistory();
```

Run कर

Expected output मध्ये शेवटी साधारण:

```
===== Transaction History =====
```

```
-----
```

```
Transaction ID : 1
```

```
Type          : Deposit
```

```
Amount        : ₹2000.0
```

```
Date & Time    : 2026-09-01T...
```

Transaction ID : 2

Type : Withdraw

Amount : ₹1500.0

Date & Time : 2026-09-01T...

महत्वाच: ₹6000 चा failed withdrawal history मध्ये दिसणार नाही, कारण transaction successful झाला नाही. ✓

आता हे run करून output पाठव. त्यानंतर आपण **Update Account + Delete Account** functionality करू.

एकदम मस्त! 🔥 **Transaction History सुद्धा perfectly काम करतेय.** ✓

तुझ्या output वरून:

- Deposit ₹2000 → Transaction ID 1 ✓
- Withdraw ₹1500 → Transaction ID 2 ✓
- Failed withdrawal ₹6000 → History मध्ये नाही ✓
- Date & Time automatically save झाला ✓
- Balance ₹5500 योग्य आहे ✓

आता project चा flow असा झाला आहे:

Banking Management System

|

| — Account

| | — Create

| | — Search

| | — Display

|

| — Banking Operations

| | — Deposit

| | — Withdraw

|

| — Exception Handling

| └─ InsufficientBalanceException

|

└─ Transaction

└─ Transaction History

पुढचा Step — Update + Delete Account

आता आपण BankService मध्ये:

updateAccount()

deleteAccount()

हे दोन methods बनवू.

उदा.:

Update:

Account 1001

Name: Reshma

Phone: 9876543210

↓ Update

Name: Reshma Patil

Phone: 9999999999

Delete:

Account 1002

↓

Delete

↓

Account deleted successfully!

इथे तुला ArrayList मधून **object update आणि remove** कसं करायचं ते शिकायला मिळेल.

पुढच्या step मध्ये BankService.java मध्ये Update आणि Delete methods add करूया. 

छान 👍 आता Update Account + Delete Account add करूया.

Step 12 — BankService.java मध्ये दोन methods add कर

service → BankService.java उघड.

सगळा code delete करू नकोस.

फक्त displayTransactionHistory() method च्या नंतर आणि शेवटच्या } च्या आधी हे दोन methods paste कर.

```
// Update Account
```

```
public void updateAccount(long accountNumber,  
    String newName,  
    String newPhoneNumber,  
    int newPin) {
```

```
    Account account = searchAccount(accountNumber);
```

```
    if (account == null) {
```

```
        System.out.println("Account not found.");
```

```
        return;
```

```
    }
```

```
    account.setAccountHolderName(newName);
```

```
    account.setPhoneNumber(newPhoneNumber);
```

```
    account.setPin(newPin);
```

```
    System.out.println("Account updated successfully!");
```

```
}
```

```
// Delete Account
```

```
public void deleteAccount(long accountNumber) {
```

```
    Account account = searchAccount(accountNumber);
```

```
    if (account == null) {
```

```
        System.out.println("Account not found.");  
        return;  
    }
```

```
accounts.remove(account);
```

```
        System.out.println("Account deleted successfully!");  
    }
```

📌 Placement असा असायला हवा

```
public void displayTransactionHistory() {
```

```
    // existing code...  
}
```

```
// Update Account
```

```
public void updateAccount(...) {
```

```
    // code...  
}
```

```
// Delete Account
```

```
public void deleteAccount(...) {
```

```
    // code...  
}
```

```
} // BankService class ची शेवटची bracket
```

शेवटची } delete करू नकोस.

Step 13 — Main.java मध्ये Update test

Main.java मध्ये Transaction History च्या खाली हे add कर:

```
System.out.println("\n===== Update Account =====");
```

```
bankService.updateAccount(  
    1001,  
    "Reshma Patil",  
    "9999999999",  
    4321  
);
```

```
Account updated = bankService.searchAccount(1001);
```

```
System.out.println("Updated Name : "  
    + updated.getAccountHolderName());
```

```
System.out.println("Updated Phone : "  
    + updated.getPhoneNumber());
```

Output:

```
===== Update Account =====
```

```
Account updated successfully!
```

```
Updated Name : Reshma Patil
```

```
Updated Phone : 9999999999
```

Step 14 — Delete test

Update test च्या खाली हे add कर:

```
System.out.println("\n===== Delete Account =====");
```

```
bankService.deleteAccount(1002);
```

```
System.out.println("\n===== Accounts After Delete =====");
```

```
bankService.displayAllAccounts();
```

Expected output:

```
===== Delete Account =====
```

Account deleted successfully!

```
===== Accounts After Delete =====
```

```
-----
```

Account Number : 1001

Account Holder : Reshma Patil

Phone Number : 9999999999

Balance : ₹5500.0

म्हणजे **Account 1002 ArrayList मधून remove** झाला. 

 **आजचा नवीन concept**

```
accounts.remove(account);
```

ArrayList मधून specific object remove करण्यासाठी वापरलं.

आता आपल्या Core Java project मध्ये जवळपास सगळे basic **CRUD operations** आले:

Create → Read/Search → Update → Delete 

आधी हे दोन methods add करून **Run** कर. Output पाठव; मग आपण पुढे **proper interactive menu (Scanner)** बनवू ज्यामुळे user स्वतः options select करू शकेल. 

एकदम भारी!  **Update आणि Delete दोन्ही successfully काम करत आहेत.** 

तुझ्या output वरून:

- Create Account → 
- Search Account → 
- Display All Accounts → 
- Deposit → 
- Withdraw → 

- Insufficient Balance Exception → 
- Transaction History → 
- Update Account → 
- Delete Account → 

आता आपला project **Core Java CRUD + Banking Operations** level वर तयार झाला आहे. 

आता एक महत्वाचा upgrade करूया

आतापर्यंत आपण Main.java मध्ये values hard-code केल्या आहेत:

```
bankService.deposit(1001, 2000);
```

Real application मध्ये user स्वतः amount enter करतो.

म्हणून आता आपण **Scanner वापरून interactive menu** बनवू:

```
===== BANKING MANAGEMENT SYSTEM =====
```

1. Create Account
2. Search Account
3. Display All Accounts
4. Deposit Money
5. Withdraw Money
6. Transaction History
7. Update Account
8. Delete Account
9. Exit

Enter your choice:

User 4 enter करेल → Deposit

User 5 enter करेल → Withdraw

User 9 enter करेल → Application बंद.

यासाठी आपण **Scanner, while loop, switch-case आणि user input validation** वापरणार आहोत.

हे झाल्यावर आपला project **फक्त testing code न राहता proper console-based Banking Management System** बनेल. 

पुढचा step: Main.java मध्ये हा interactive menu बनवूया.

चला 👍 आता आपल्या project ला **proper interactive Banking Management System** बनवूया.
आतापर्यंतचा hard-coded testing code आपण काढून user-input based menu बनवू.

Step 13 — Main.java पूर्ण replace कर

Main.java उघड आणि **सगळा जुना code delete करून** खालील code paste कर:

```
import exception.InsufficientBalanceException;

import model.Account;

import service.BankService;


import java.util.Scanner;

public class Main {

    public static void main(String[] args) {

        Scanner scanner = new Scanner(System.in);

        BankService bankService = new BankService();

        int choice;

        do {

            System.out.println("\n=====");
            System.out.println("  BANKING MANAGEMENT SYSTEM");
            System.out.println("=====");

            System.out.println("1. Create Account");
            System.out.println("2. Search Account");
            System.out.println("3. Display All Accounts");
```

```
System.out.println("4. Deposit Money");
System.out.println("5. Withdraw Money");
System.out.println("6. Transaction History");
System.out.println("7. Update Account");
System.out.println("8. Delete Account");
System.out.println("9. Exit");

System.out.print("\nEnter your choice: ");

choice = scanner.nextInt();

switch (choice) {

    case 1:

        System.out.println("\n===== Create Account =====");

        System.out.print("Enter Account Number: ");
        long accountNumber = scanner.nextLong();

        scanner.nextLine();

        System.out.print("Enter Account Holder Name: ");
        String name = scanner.nextLine();

        System.out.print("Enter Phone Number: ");
        String phone = scanner.nextLine();

        System.out.print("Enter PIN: ");
        int pin = scanner.nextInt();
```

```
System.out.print("Enter Initial Balance: ");  
double balance = scanner.nextDouble();
```

```
Account account = new Account(  
    accountNumber,  
    name,  
    phone,  
    pin,  
    balance  
);
```

```
bankService.createAccount(account);
```

```
break;
```

case 2:

```
System.out.println("\n===== Search Account =====");
```

```
System.out.print("Enter Account Number: ");  
long searchNumber = scanner.nextLong();
```

```
Account foundAccount =  
    bankService.searchAccount(searchNumber);
```

```
if (foundAccount != null) {
```

```
    System.out.println("Account Found!");  
    System.out.println("Account Number : "  
        + foundAccount.getAccountNumber());
```



```
System.out.println("Account Holder : "  
    + foundAccount.getAccountHolderName());
```

```
System.out.println("Phone Number  : "  
    + foundAccount.getPhoneNumber());
```

```
System.out.println("Balance      : ₹"  
    + foundAccount.getBalance());
```

```
} else {
```

```
    System.out.println("Account not found.");  
}
```

```
break;
```

```
case 3:
```

```
System.out.println("\n===== All Accounts =====");
```

```
bankService.displayAllAccounts();
```

```
break;
```

```
case 4:
```

```
System.out.println("\n===== Deposit Money =====");
```

```
System.out.print("Enter Account Number: ");  
long depositAccountNumber = scanner.nextLong();
```

```
System.out.print("Enter Deposit Amount: ");  
double depositAmount = scanner.nextDouble();
```

```
bankService.deposit(  
    depositAccountNumber,  
    depositAmount  
);
```

```
break;
```

case 5:

```
System.out.println("\n===== Withdraw Money =====");
```

```
System.out.print("Enter Account Number: ");  
long withdrawAccountNumber = scanner.nextLong();
```

```
System.out.print("Enter Withdrawal Amount: ");  
double withdrawAmount = scanner.nextDouble();
```

```
try {
```

```
    bankService.withdraw(  
        withdrawAccountNumber,  
        withdrawAmount  
    );
```

```
} catch (InsufficientBalanceException e) {
```

```
    System.out.println(e.getMessage());
```

```
}
```

```
break;
```

case 6:

```
System.out.println("\n===== Transaction History =====");
```

```
bankService.displayTransactionHistory();
```

```
break;
```

case 7:

```
System.out.println("\n===== Update Account =====");
```

```
System.out.print("Enter Account Number: ");
```

```
long updateAccountNumber = scanner.nextLong();
```

```
scanner.nextLine();
```

```
System.out.print("Enter New Name: ");
```

```
String newName = scanner.nextLine();
```

```
System.out.print("Enter New Phone Number: ");
```

```
String newPhone = scanner.nextLine();
```

```
System.out.print("Enter New PIN: ");  
int newPin = scanner.nextInt();
```

```
bankService.updateAccount(  
    updateAccountNumber,  
    newName,  
    newPhone,  
    newPin  
);
```

```
break;
```

case 8:

```
System.out.println("\n===== Delete Account =====");
```

```
System.out.print("Enter Account Number: ");  
long deleteAccountNumber = scanner.nextLong();
```

```
bankService.deleteAccount(deleteAccountNumber);
```

```
break;
```

case 9:

```
System.out.println("\nThank you for using Banking Management System!");
```

```
break;
```

default:

```
        System.out.println("\nInvalid choice! Please try again.");
    }

    } while (choice != 9);

    scanner.close();
}
}
```

आता काय बदललं?

आधी आपण असं manually करत होतो:

```
bankService.deposit(1001, 2000);
```

आता user स्वतः enter करेल:

Enter Account Number: 1001

Enter Deposit Amount: 2000

आणि:

```
Scanner scanner = new Scanner(System.in);
```

user कडून input घेण्यासाठी वापरला.

switch-case user चा option handle करतो आणि:

```
do {
    ...
} while (choice != 9);
```

मुळे menu पुन्हा पुन्हा दिसत राहतो, जोपर्यंत user 9 निवडत नाही.

आता Test करूया

Run केल्यावर सुरुवातीला:

=====

BANKING MANAGEMENT SYSTEM

=====

1. Create Account
2. Search Account
3. Display All Accounts
4. Deposit Money
5. Withdraw Money
6. Transaction History
7. Update Account
8. Delete Account
9. Exit

Enter your choice:

पहिली test

1 enter कर.

नंतर:

Account Number: 1001

Account Holder Name: Reshma

Phone Number: 9876543210

PIN: 1234

Initial Balance: 5000

मग 4 वापरून **Deposit**, 5 वापरून **Withdraw**, आणि 6 वापरून **Transaction History** check कर.

⚠ एक गोष्ट लक्षात ठेव

आता program बंद केला आणि पुन्हा Run केला तर accounts गायब होतील.

कारण data सध्या फक्त **ArrayList** मध्ये **RAM** मध्ये आहे.

पुढच्या major step मध्ये आपण **File Handling** वापरून accounts/data file मध्ये save करू. म्हणजे application बंद करून पुन्हा सुरू केल्यावरही data available राहील. 💾

आधी हा Main.java run करून menu दिसतोय का ते check कर.

एकदम **perfect!** 🔥 तुझं interactive Banking Management System व्यवस्थित चालतंय. काही error नाही. ✅

तुझ्या testing मध्ये आपण खरंच complete flow करून पाहिला:

Initial Balance	₹5,000
Deposit	₹5,000
Balance	₹10,000
Deposit	₹4,000
Balance	₹14,000
Withdraw	₹5,000
Balance	₹9,000
Withdraw	₹4,000
Final Balance	₹5,000

आणि Transaction History मध्ये चारही successful transactions आल्या आहेत. **100**

आता एक महत्वाची गोष्ट

तुझा program सध्या या line वर थांबलेला आहे:

Enter your choice:

हे बरोबर आहे. Program अजून चालू आहे आणि तुझ्या पुढच्या input ची वाट पाहतोय.

Testing संपवायचं असेल तर:

9

enter कर.

मग:

Thank you for using Banking Management System!

आणि program बंद होईल.

🔗 आता आपला Core Java project कुठे पोहोचला?

आपल्याकडे आता:

Core Java

- ✅ Classes & Objects

- ☒ Constructors
- ☒ Encapsulation
- ☒ Getters/Setters
- ☒ Packages
- ☒ ArrayList / Collections
- ☒ Loops
- ☒ switch-case
- ☒ Scanner
- ☒ Exception Handling
- ☒ Custom Exception
- ☒ try-catch
- ☒ throw / throws
- ☒ LocalDateTime
- ☒ CRUD operations
- ☒ Transaction management

आता फक्त एक **खूप महत्वाचा Core Java topic — File Handling** बाकी आहे.

कारण सध्या program बंद केल्यावर:

Run program



Create Account



Deposit



Data in ArrayList



Program बंद ❌



Data गायब

आपल्याला हे बदलायचं आहे:

Run program



Create / Deposit / Withdraw



Data



Save to File 



Program बंद



Program पुन्हा Run



Data पुन्हा Load 

यामुळे तुझ्या Core Java project मध्ये File Handling चा real-world use सुद्धा येईल.

पुढचा step: File Handling वापरून Account data save/load करूया.  

छान 👍 आता आपण आपल्या Core Java project मधला File Handling भाग करूया. पण हा भाग थोडा मोठा आहे, म्हणून आपण हळूहळू करू.

Step 14 — Account data file मध्ये save करणे 

आता ArrayList मध्ये accounts आहेत, पण program बंद झाला की ते disappear होतात.

आपण FileUtil.java बनवून data file मध्ये save करू.

1 util package तयार कर

IntelliJ मध्ये:

src → Right Click → New → Package

नाव:

util

नंतर:

util → Right Click → New → Java Class

नाव:

2 FileUtil.java मध्ये हा code टाक

```
package util;

import model.Account;

import java.io.FileWriter;
import java.io.IOException;
import java.util.ArrayList;

public class FileUtil {

    private static final String FILE_NAME = "accounts.txt";

    // Save accounts to file
    public static void saveAccounts(ArrayList<Account> accounts) {

        try (FileWriter writer = new FileWriter(FILE_NAME)) {

            for (Account account : accounts) {

                writer.write(
                    account.getAccountNumber() + "," +
                    account.getAccountHolderName() + "," +
                    account.getPhoneNumber() + "," +
                    account.getPin() + "," +
                    account.getBalance() +
                    "\n"
                );
            }
        }
    }
}
```

```
        System.out.println("Accounts saved successfully!");

    } catch (IOException e) {

        System.out.println("Error while saving accounts.");
    }
}
}
```

🧐 हा code काय करतो?

ही line:

```
private static final String FILE_NAME = "accounts.txt";
```

आपण data **accounts.txt** नावाच्या file मध्ये save करणार आहोत.

उदा. file मध्ये data असा दिसू शकतो:

```
1001,Reshma,9876543210,1234,5000.0
```

```
1002,Rahul,9876543211,5678,10000.0
```

आणि:

```
try (FileWriter writer = new FileWriter(FILE_NAME))
```

File open करून त्यामध्ये data लिहितो.

⚠ पण एक महत्वाची गोष्ट

आता आपण फक्त **FileUtil** तयार केला आहे.

तो automatically काम करणार नाही, कारण **BankService** मधून त्याला call करावं लागेल.

आणि त्यासाठी **BankService** मधल्या accounts list ला **FileUtil** पर्यंत access द्यावा लागेल.

हे आपण पुढच्या step मध्ये करू.

आता फक्त **FileUtil.java** तयार करून code paste कर आणि Save कर. Run करू नकोस.

झाल्यावर मला “**FileUtil तयार झाला**” असं सांग.

छान 👍 **FileUtil.java** तयार झाला आहे. आता आपण **FileUtil** ला **BankService** सोबत connect करू.

आता आपण एकच काम करू: **Account create** केल्यावर **accounts.txt** मध्ये **data save** होईल.

Step 15 — BankService.java update

service → BankService.java उघड.

1. वरचा import add कर

सध्या imports असे आहेत:

```
import exception.InsufficientBalanceException;
```

```
import model.Account;
```

```
import model.Transaction;
```

```
import java.util.ArrayList;
```

यामध्ये हा import add कर:

```
import util.FileUtil;
```

म्हणजे:

```
import exception.InsufficientBalanceException;
```

```
import model.Account;
```

```
import model.Transaction;
```

```
import util.FileUtil;
```

```
import java.util.ArrayList;
```

2. createAccount() method शोध

सध्या तुझ्याकडे:

```
public void createAccount(Account account) {
```

```
    accounts.add(account);
```

```
    System.out.println("Account created successfully!");
```

```
}
```

हे पूर्ण **replace** करून:

```
public void createAccount(Account account) {
```

```
accounts.add(account);
```

```
FileUtil.saveAccounts(accounts);
```

```
System.out.println("Account created successfully!");
```

```
}
```

🧠 आता काय होईल?

User:

1. Create Account

करतो.

मग:

Account

↓

accounts ArrayList मध्ये add

↓

FileUtil.saveAccounts()

↓

accounts.txt 💾

उदा. account create केल्यावर project folder मध्ये:

accounts.txt

file तयार होईल.

त्यात:

1001,Reshma,9876543210,1234,5000.0

असं data दिसेल.

⚠️ आता FileUtil.java मध्ये काही बदल करू नकोस.

फक्त:

1. BankService.java मध्ये import util.FileUtil; add कर.

2. createAccount() method replace कर.
3. **Ctrl + S**
4. Run कर.
5. 1 → Create Account करून एखादा account create कर.

मग IntelliJ project मध्ये accounts.txt तयार झाली का ते बघ.

जर file तयार झाली आणि त्यात account data दिसला, तर आपण पुढच्या step मध्ये **program पुन्हा सुरू केल्यावर file मधून accounts automatically load करू**.   

हो 👍 अगदी सोप्या पद्धतीने सांगतो. **Step 15 मधला फक्त एक छोटा बदल** करायचा आहे.

1 BankService.java उघड

त्यात हा भाग शोध:

```
public void createAccount(Account account) {  
  
    accounts.add(account);  
  
    System.out.println("Account created successfully!");  
}
```

2 हा पूर्ण भाग delete कर

वरील पूर्ण createAccount method delete कर.

3 त्याच जागी हा code paste कर

```
public void createAccount(Account account) {  
  
    accounts.add(account);  
  
    FileUtil.saveAccounts(accounts);  
  
    System.out.println("Account created successfully!");  
}
```

फरक फक्त एक line आहे:

```
FileUtil.saveAccounts(accounts);
```

ही line accounts.add(account); नंतर आली आहे.

4 वर import आहे का ते check कर

BankService.java च्या अगदी वर:

```
import util.FileUtil;
```

हे असायलाच पाहिजे.

म्हणजे सुरुवातीचा भाग असा:

```
package service;
```

```
import exception.InsufficientBalanceException;
```

```
import model.Account;
```

```
import model.Transaction;
```

```
import util.FileUtil;
```

```
import java.util.ArrayList;
```

बस! आता बाकी BankService.java मधला कोणताही code बदलायचा नाही.

Save करून Run कर. मग 1 निवडून account create कर. accounts.txt तयार झाली का ते पाहू.

हो! 🎉 **Step 15 successfully झाला आहे.** ✅

तुझ्या output मध्ये:

```
Accounts saved successfully!
```

```
Account created successfully!
```

हे दिसतंय म्हणजे FileUtil.saveAccounts(accounts) नीट call झालं आहे.

आता एकदा accounts.txt शोध

IntelliJ च्या **Project** window मध्ये project folder पाहा.

साधारण:

```
BankingManagementSystem
```

```
|
```

```
|— src
```

```
| ├── model
| ├── service
| ├── exception
| └── util
|
|── accounts.txt ← ही file
└── ...
```

accounts.txt उघडल्यावर साधारण:

1001,Reshma,11111,12345,10000.0

असं दिसायला पाहिजे. 

 आता program बंद करायचा असेल

Console मध्ये:

Enter your choice:

इथे:

9

टाक.

पुढचा Step   

आता data **save** होत आहे. पण अजून एक problem आहे:

Program Start

↓

ArrayList रिकामी 

↓

accounts.txt मध्ये data आहे

म्हणून पुढे आपण **loadAccounts()** बनवू.

त्यामुळे:

Program Start

↓

accounts.txt



loadAccounts()



ArrayList मध्ये accounts



Search / Deposit / Withdraw

म्हणजे program बंद करून पुन्हा चालू केला तरी account data राहील. 🚀

आता accounts.txt मध्ये वरचा data दिसतोय का ते check कर.

छान 👍 आता Step 16 — File मधून Account data परत load करणे करूया.

यामुळे program पुन्हा Run केल्यावर account manually create करावा लागणार नाही.

1 FileUtil.java उघड

आता त्यात saveAccounts() method आहे. त्याच्या खाली, शेवटच्या } च्या आधी हा method add कर:

```
public static ArrayList<Account> loadAccounts() {
```

```
    ArrayList<Account> accounts = new ArrayList<>();
```

```
    java.io.File file = new java.io.File(FILE_NAME);
```

```
    if (!file.exists()) {
```

```
        return accounts;
```

```
    }
```

```
    try (java.util.Scanner scanner = new java.util.Scanner(file)) {
```

```
        while (scanner.hasNextLine()) {
```

```
            String line = scanner.nextLine();
```

```

String[] data = line.split(",");

long accountNumber = Long.parseLong(data[0]);
String name = data[1];
String phone = data[2];
int pin = Integer.parseInt(data[3]);
double balance = Double.parseDouble(data[4]);

Account account = new Account(
    accountNumber,
    name,
    phone,
    pin,
    balance
);

accounts.add(account);
}

System.out.println("Accounts loaded successfully!");

} catch (Exception e) {

    System.out.println("Error while loading accounts.");
}

return accounts;
}

```

2 आता BankService.java मध्ये

accounts declaration शोध:

```
private ArrayList<Account> accounts = new ArrayList<>();
```

ही line delete करू नकोस. तिच्या खाली हा constructor add कर:

```
public BankService() {  
  
    accounts = FileUtil.loadAccounts();  
}
```

म्हणजे:

```
private ArrayList<Account> accounts = new ArrayList<>();
```

```
public BankService() {  
  
    accounts = FileUtil.loadAccounts();  
}
```

 आता flow असा होईल

Program सुरू होताना:

Main



BankService()



FileUtil.loadAccounts()



accounts.txt



ArrayList मध्ये Account

म्हणजे आधी save केलेला:

1001,Reshma,11111,12345,10000.0

पुन्हा Account object मध्ये convert होईल. 

 Test

आधी current program मध्ये 9 देऊन बंद कर.

नंतर पुन्हा Run कर.

काहीही Create करू नकोस.

सुरुवातीला:

Accounts loaded successfully!

=====

BANKING MANAGEMENT SYSTEM

=====

नंतर 3 निवड:

===== All Accounts =====

Account Number : 1001

Account Holder : Reshma

Phone Number : 11111

Balance : ₹10000.0

असं आलं तर File Handling Save + Load successfully complete झालं आहे. 🎉

Run करून output पाठव.

हो, हा error बहुतेक FileUtil.java मध्ये नवीन method चुकीच्या { } bracket च्या आत/बाहेर paste झाल्यामुळे आला आहे. काळजी करू नकोस. आपण FileUtil.java एकदम clean करूया. 👍

Step 1 — FileUtil.java पूर्ण replace कर

util → FileUtil.java उघड.

आतला पूर्ण code delete कर आणि खालील code जसाच्या तसा paste कर:

```
package util;
```

```
import model.Account;
```

```
import java.io.File;
```

```
import java.io.FileWriter;
```

```
import java.util.ArrayList;
```

```
import java.util.Scanner;

public class FileUtil {

    private static final String FILE_NAME = "accounts.txt";

    // Save Accounts
    public static void saveAccounts(ArrayList<Account> accounts) {

        try (FileWriter writer = new FileWriter(FILE_NAME)) {

            for (Account account : accounts) {

                writer.write(
                    account.getAccountNumber() + "," +
                    account.getAccountHolderName() + "," +
                    account.getPhoneNumber() + "," +
                    account.getPin() + "," +
                    account.getBalance() +
                    "\n"
                );
            }

            System.out.println("Accounts saved successfully!");

        } catch (Exception e) {

            System.out.println("Error while saving accounts.");
        }
    }
}
```

```
// Load Accounts

public static ArrayList<Account> loadAccounts() {

    ArrayList<Account> accounts = new ArrayList<>();

    File file = new File(FILE_NAME);

    if (!file.exists()) {
        return accounts;
    }

    try (Scanner scanner = new Scanner(file)) {

        while (scanner.hasNextLine()) {

            String line = scanner.nextLine();

            String[] data = line.split(",");

            long accountNumber =
                Long.parseLong(data[0]);

            String name = data[1];

            String phone = data[2];

            int pin =
                Integer.parseInt(data[3]);
```

```

        double balance =
            Double.parseDouble(data[4]);

        Account account = new Account(
            accountNumber,
            name,
            phone,
            pin,
            balance
        );

        accounts.add(account);
    }

    System.out.println("Accounts loaded successfully!");

} catch (Exception e) {

    System.out.println("Error while loading accounts.");
}

return accounts;
}
}

```

Step 2 — brackets check कर

FileUtil.java च्या शेवटी **फक्त एक }** असायला पाहिजे, जो class FileUtil बंद करतो:

```

return accounts;
}
}

```

म्हणजे:

- पहिला } → loadAccounts() बंद
 - दुसरा } → FileUtil class बंद
-

Step 3 — BankService.java check कर

वर हे असायलाच पाहिजे:

```
import util.FileUtil;
```

आणि:

```
private ArrayList<Account> accounts = new ArrayList<>();
```

```
public BankService() {
```

```
    accounts = FileUtil.loadAccounts();
```

```
}
```

हा constructor BankService class च्या आत असला पाहिजे.

Step 4 — आता लगेच Run करू नकोस

आधी IntelliJ मध्ये:

Build → Rebuild Project

कर.

जर Rebuild successful झालं, मग Main.java Run कर.

अपेक्षित result

तुझ्या आधीच्या accounts.txt मध्ये account save झालेला आहे, त्यामुळे program सुरू करताना:

Accounts loaded successfully!

आणि menu दिसेल.

मग 3 निवडल्यावर:

===== All Accounts =====

Account Number : 1001

Account Holder : Reshma

Phone Number : 11111

Balance : ₹10000.0

दिसायला पाहिजे. 

आता फक्त FileUtil.java पूर्ण replace कर → Rebuild Project कर. जर पुन्हा red error आला तर FileUtil.java चा screenshot किंवा पूर्ण code पाठव; आपण bracket कुठे चुकली आहे ते नेमकं पाहू.

हो  IntelliJ IDEA मध्ये Rebuild Project करण्याचे exact steps सांगतो.

IntelliJ मध्ये:

1. वरच्या menu bar मध्ये **Build** वर click कर.
2. आलेल्या menu मध्ये **Rebuild Project** शोध.
3. त्यावर click कर.

मग खाली **Build** window मध्ये process चालेल.

जर सगळं ठीक असेल तर शेवटी साधारण:

BUILD SUCCESSFUL

किंवा:

Build completed successfully

असं दिसेल. 

जर Rebuild Project दिसत नसेल

Build menu चा screenshot मला पाठव. मी तुला कुठे click करायचं ते screenshot वरून exact सांगतो.

हो  Screenshot पाहिला. तुझा Build successful झालेला आहे! 

खाली Build Output मध्ये स्पष्ट दिसतंय:

BankingManagementSystem: build finished

म्हणजे आता Build/Rebuild मध्ये error नाही. 🎉

आता पुढचं काय करायचं?

तुझ्या screenshot मध्ये FileUtil.java चा शेवटही योग्य दिसतोय:

```
return accounts;
}
}
```

म्हणून FileUtil.java ची bracket problem आता fix झाली आहे. 👍

आता Step 16 test करूया

1. Main.java वर जा.
2. वरचा हिरवा Run ► button click कर.
3. Program सुरू झाल्यावर काहीही Create करू नकोस.
4. जर सुरुवातीला:

Accounts loaded successfully!

असं आलं, तर loading successful आहे. ✅

5. मग menu मध्ये:

Enter your choice: 3

टाक.

मग तुझा आधी save केलेला account दिसायला पाहिजे:

===== All Accounts =====

Account Number : 1001

Account Holder : Reshma

Phone Number : 11111

Balance : ₹10000.0

⚠️ एक महत्वाची गोष्ट

तुझ्या screenshot मध्ये accounts.txt tab दिसतोय, म्हणजे file project मध्ये आहे. 👍

आता Run करून जे output येईल ते मला पाठव. मग आपण खात्री करू की Save + Load दोन्ही successfully झाले आहेत. 💾 ✅

हो 👍 Screenshot पाहिला. काही error नाही. तुझा Main program आधीपासून run होत आहे.

आता IntelliJ हा message दाखवत आहे:

Process 'Main' Is Running

म्हणजे तू पुन्हा Run दाबलंस, पण पहिला Main अजून बंद झालेला नाही.

आता काय करायचं?

Popup मध्ये **Stop and Rerun** button आहे.

👉 त्यावर click कर.

मग जुना program बंद होईल आणि नवीन program सुरू होईल. ✅

नवीन Run सुरू झाल्यावर

Console मध्ये:

BANKING MANAGEMENT SYSTEM

आणि:

1. Create Account
2. Search Account
3. Display All Accounts
- ...
9. Exit

Enter your choice:

असं दिसेल.

आता 1 निवडू नकोस.

पहिल्यांदा:

3

टाक.

कारण आपण check करत आहोत की accounts.txt मधला account program सुरू होताना **automatically load झाला आहे का.**

जर सगळं योग्य असेल तर आधी:

Accounts loaded successfully!

आणि 3 नंतर:

===== All Accounts =====

Account Number : 1001

Account Holder : Reshma

Phone Number : 11111

Balance : ₹10000.0

असं दिसेल. 

Stop and Rerun → मग 3 enter कर → आलेला output मला पाठव.

हो  आता program नवीन Run झालेला आहे आणि कोणताही error दिसत नाही. 

आता खाली:

Enter your choice:

दिसत आहे.

आता फक्त 3 टाईप कर:

3

आणि Enter दाब.

कारण 3 = Display All Accounts.

आपण तपासणार आहोत की आधी accounts.txt मध्ये save केलेला:

1001, Reshma, 11111, 12345, 10000

हा account पुन्हा file मधून load झाला आहे का.

योग्य output असा काहीतरी दिसेल:

Accounts loaded successfully!

===== All Accounts =====

Account Number : 1001

Account Holder : Reshma

Phone Number : 11111

Balance : ₹10000.0

आता 1 किंवा दुसरा option देऊ नकोस—फक्त 3 दे. मग आलेला output पाठव. 👍

हो 👍 आता कारण समजलं. Program चालतोय, पण accounts.txt मधला data load होत नाहीये. ❌

कारण output मध्ये:

Accounts loaded successfully!

हे आलेलं नाही.

बहुतेक accounts.txt वेगळ्या folder मध्ये तयार झाली आहे, आणि Java दुसऱ्या location मधून ती शोधत आहे.

काळजी करू नकोस. आपण हे व्यवस्थित fix करूया.

Step 16A — आधी accounts.txt तपासूया

तुझ्या IntelliJ मध्ये वर accounts.txt tab दिसतोय.

👉 त्या accounts.txt tab वर click कर.

त्यात हे असायला पाहिजे:

1001,Reshma,11111,12345,10000.0

जर हे data दिसत असेल ✅

मला फक्त सांग:

accounts.txt मध्ये data आहे

मग मी तुला पुढचा छोटा बदल सांगतो, ज्यामुळे Java योग्य file location मधून data load करेल.

जर accounts.txt रिकामी असेल ❌

तर मला सांग:

accounts.txt रिकामी आहे

मग आधी save कुठे होतंय ते fix करू.

आता accounts.txt उघडून त्यात काय दिसतं ते सांग.

छान 👍 म्हणजे accounts.txt मध्ये data सुरक्षित आहे. 💾

आता problem BankService मधून loadAccounts() call होत नाहीये किंवा योग्य ठिकाणी नाहीये.

आपण अंदाजाने काही delete करणार नाही. फक्त BankService.java चा एक छोटा भाग check करूया.

Step 16B — BankService.java उघड

BankService.java मध्ये अगदी वरचा भाग पाहा.

तिथे हे दोन भाग आहेत का ते check कर:

```
private ArrayList<Account> accounts = new ArrayList<>();
```

```
public BankService() {
```

```
    accounts = FileUtil.loadAccounts();
```

```
}
```

आणि वर imports मध्ये:

```
import util.FileUtil;
```

जर हे तीनही भाग आहेत

मला BankService.java चा पूर्ण code इथे paste कर.

जर constructor हा भाग नाहीये

तर private ArrayList... च्या लगेच खाली:

```
public BankService() {
```

```
    accounts = FileUtil.loadAccounts();
```

```
}
```

paste कर.

बाकी BankService.java मधलं काहीही delete करू नकोस.

मग Save → Run → 3 करून पाहू.

यावेळी अपेक्षित आहे:

Accounts loaded successfully!

```
===== All Accounts =====
```

```
-----
```

Account Number : 1001

Account Holder : Reshma

Phone Number : 11111

Balance : ₹10000.0

जर BankService.java मध्ये नेमकं कुठे paste करायचं हे कळत नसेल, त्याचा screenshot पाठव—मी तुझ्या code वरून exact जागा सांगतो.

हो 👍 आता exact problem दिसला.

तुझ्या BankService.java मध्ये:

```
private ArrayList<Account> accounts = new ArrayList<>();
```

आहे, पण **BankService() constructor नाहीये**. त्यामुळे accounts.txt मधला data load होत नाही.

फक्त हे कर 🙌

या line नंतर:

```
private ArrayList<Account> accounts = new ArrayList<>();
```

हा code paste कर:

```
public BankService() {
```

```
    accounts = FileUtil.loadAccounts();
```

```
}
```

म्हणजे सुरुवातीचा भाग असा दिसला पाहिजे:

```
public class BankService {
```

```
    private ArrayList<Account> accounts = new ArrayList<>();
```

```
    public BankService() {
```

```
        accounts = FileUtil.loadAccounts();
```

```
    }
```

```
    private ArrayList<Transaction> transactions = new ArrayList<>();
```

```
private int transactionCounter = 1;
```

⚠ बाकी काहीही delete करू नकोस.

तुझ्याकडे import util.FileUtil; आधीपासून आहे, त्यामुळे ते पुन्हा add करण्याची गरज नाही. ✅

मग:

1. **Ctrl + S**
2. Run करण्यासाठी हिरवा ► button दाब.
3. Program सुरू झाल्यावर:

3

enter कर.

आता expected output:

Accounts loaded successfully!

```
=====
BANKING MANAGEMENT SYSTEM
=====
```

...

Enter your choice: 3

```
===== All Accounts =====
```

```
-----
```

Account Number : 1001

Account Holder : Reshma

Phone Number : 11111

Balance : ₹10000.0

हे आलं तर 🎉 File Save + File Load पूर्णपणे working आहे.

एक छोटा पुढचा मुद्दा: Deposit/Withdraw/Update/Delete झाल्यानंतर सुद्धा file मध्ये नवीन balance save व्हायला हवा. सध्या फक्त Create केल्यावर save होतं. ते आपण लगेच पुढच्या step मध्ये fix करू.

हो! 🎉 आता File Handling चा Save + Load दोन्ही successfully झाले आहेत. ✅

तुझ्या output मध्ये:

Accounts loaded successfully!

आणि 3 केल्यावर:

Account Number : 1001

Account Holder : Reshma

Phone Number : 11111

Balance : ₹10000.0

म्हणजे:

Program बंद → पुन्हा Run → accounts.txt मधून account परत आला ✅ 💾

आता एक महत्वाचा bug fix करूया

सध्या Deposit / Withdraw केल्यावर balance फक्त ArrayList मध्ये बदलतो, पण accounts.txt मध्ये नवीन balance save होत नाही.

उदा.:

Current file balance = ₹10000

↓

Deposit ₹2000

↓

Program मध्ये = ₹12000 ✅

accounts.txt = ₹10000 ❌

Program पुन्हा सुरू केला तर पुन्हा ₹10000 दिसेल.

म्हणून आता Deposit, Withdraw, Update आणि Delete नंतर FileUtil.saveAccounts(accounts); करायचं.

पुढच्या step मध्ये हे चारही operations एकत्र fix करूया. त्यानंतर आपला Core Java project अधिक complete होईल. 🚀

छान 📌 आता **Step 17 — Deposit/Withdraw/Update/Delete नंतर data file मध्ये save** करूया.

इथे पूर्ण BankService.java बदलायची गरज नाही. फक्त 4 ठिकाणी एक line add करायची आहे.

1 Deposit मध्ये

deposit() method मध्ये हे आहे:

```
account.setBalance(newBalance);
```

त्याच्या लगेच खाली add कर:

```
FileUtil.saveAccounts(accounts);
```

म्हणजे:

```
account.setBalance(newBalance);
```

```
FileUtil.saveAccounts(accounts);
```

2 Withdraw मध्ये

withdraw() मध्ये हे आहे:

```
account.setBalance(newBalance);
```

त्याच्या खाली:

```
FileUtil.saveAccounts(accounts);
```

म्हणजे:

```
account.setBalance(newBalance);
```

```
FileUtil.saveAccounts(accounts);
```

3 Update Account मध्ये

तुझ्याकडे:

```
account.setAccountHolderName(newName);
```

```
account.setPhoneNumber(newPhone);
```

```
account.setPin(newPin);
```

```
System.out.println("Account updated successfully!");
```

यामध्ये setPin() नंतर ही line add कर:

```
FileUtil.saveAccounts(accounts);
```

म्हणजे:

```
account.setAccountHolderName(newName);
```

```
account.setPhoneNumber(newPhone);
```

```
account.setPin(newPin);
```

```
FileUtil.saveAccounts(accounts);
```

```
System.out.println("Account updated successfully!");
```

Delete Account मध्ये

तुझ्याकडे:

```
accounts.remove(account);
```

```
System.out.println("Account deleted successfully!");
```

यामध्ये remove() नंतर ही line add कर:

```
FileUtil.saveAccounts(accounts);
```

म्हणजे:

```
accounts.remove(account);
```

```
FileUtil.saveAccounts(accounts);
```

```
System.out.println("Account deleted successfully!");
```

आता Test करूया

सध्याच्या program ला 9 देऊन बंद कर.

मग पुन्हा Run कर.

Test 1 — Deposit

3 → balance check कर.

नंतर:

4

उदा.:

Account Number: 1001

Deposit Amount: 2000

मग 9 करून program बंद कर.

Test 2 — पुन्हा Run

Program पुन्हा सुरू केल्यावर:

3

केल्यावर balance:

₹12000.0

दिसला पाहिजे. 

याचा अर्थ:

Deposit

↓

ArrayList update

↓

accounts.txt update 

↓

Program restart

↓

Data वापस load 

हे test करून output पाठव. मग आपण Core Java project चा शेवटचा major भाग — validation + duplicate account checking — करू. 